

System Design for Modern **Microservices**

Principles | Patterns | Practices

Ravi Sajjanar

Cloud Developer - 2

HPE

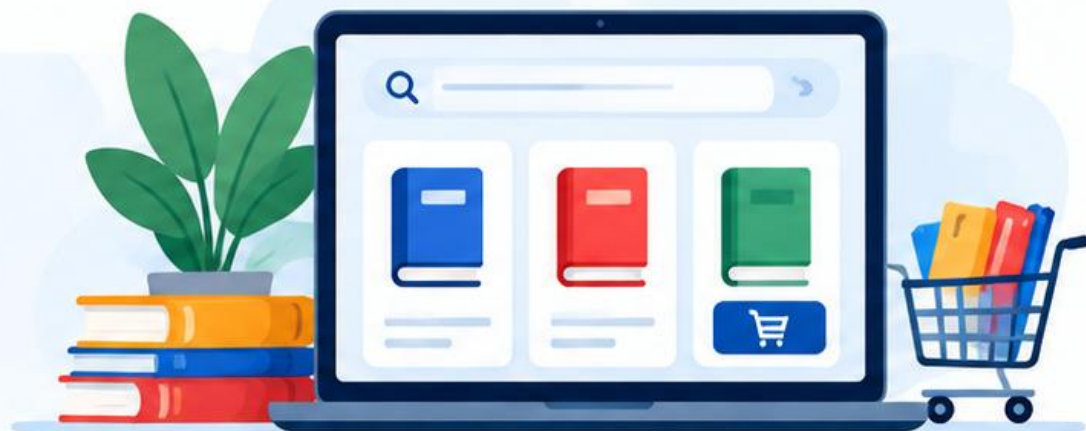


Session Objective

By the end of this session, you should be able to take a real-world problem like:

“Design an **online bookstore application.”**

and confidently go from:



Requirements

Identify key features for the bookstore.



Architecture

Design high-level and low-level architecture.



Microservices

Identify services (e.g., catalog, orders, user).



Database

Choose data models and storage strategy.



Scaling

Handle increasing traffic and high load.



Reliability & Trade-offs

Plan for failures, availability and make trade-offs.

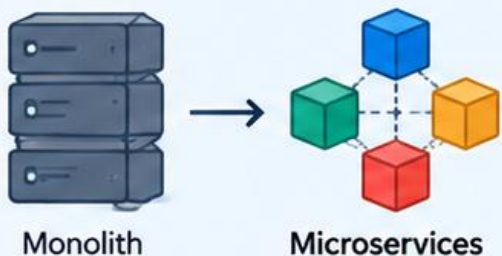


While being able to **explain why** you made each decision.

Session Agenda

1

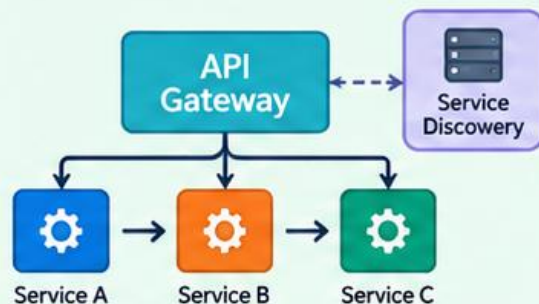
Microservices Fundamentals



- What are microservices?
- Monolith vs. microservices
- Service boundaries
- Principles of scalable cloud-native system design

2

Microservices Architecture Patterns



- API Gateway
- Service Discovery
- Event-Driven Architecture
- Inter-service communication
- Designing for resilience and scalability

3

Containerization & Orchestration



- Docker fundamentals
- Kubernetes concepts
- Deployment patterns
- Scaling and self-healing
- Cloud-native application architecture

4

Hands-on & Industry Insights



- Build and deploy a microservices-based application
- Security best practices
- Real-world case studies
- Practical design considerations
- Industry insights and Q&A

Session Flow



Understand
(Fundamentals)



Design
(Architecture Patterns)



Containerize & Orchestrate
(Docker + Kubernetes)



Build & Deploy
(Hands-on + Insights)

Think Like a System Designer



Design → Build → Deploy → Scale



The goal is not to memorize architectures. The goal is to learn how to arrive at an architecture.

BookStore – the workshop application

An online bookstore where users can browse, search and **buy books**.



Where do we store the **books, users and orders**?



How many **users** can it support?



How do users **search and find books**?



How is the **catalog managed** and kept up to date?



How do we ensure **security and data privacy** (e.g., user data, payments)?



How do we handle **orders and payments**?



How do we **scale** when thousands of users visit at the same time?



What happens if a **service** (e.g., catalog or payment) goes down?



System design is the process of taking a **real-world requirement** (like an online bookstore) and converting it into a **technical architecture** that can satisfy those needs at scale.

Steps in System Design

From a problem to a production-ready system

1

Understand the Problem



- What are we building?
- Who are the users?
- What are the core use cases?

Example:
Online BookStore

2

Define Requirements



- Functional requirements
- Non-functional requirements (performance, availability, security, etc.)
- Constraints and assumptions

3

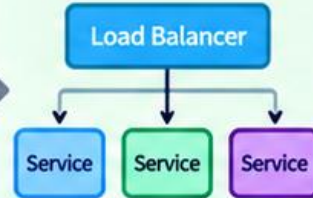
Estimate Scale



- Number of users
- Requests per second
- Data volume
- Read/Write ratio
- Traffic growth

4

Design High-Level Architecture



- Identify major components/services
- How they communicate
- Sync vs Async
- Overall architecture diagram

5

Design Components & Data



- Microservice boundaries
- APIs
- Database / data model
- Cache
- Message queues / events
- Storage

6

Design for Scale, Reliability & Resiliency



- Load balancing
- Horizontal scaling
- Replication
- Failover
- Timeouts, retries, circuit breakers
- Monitoring & alerting
- Disaster recovery

7

Evaluate Trade-offs



- Why SQL vs NoSQL?
- Why REST vs events?
- Why cache?
- Why this service boundary?
- Cost vs performance
- Complexity vs benefit



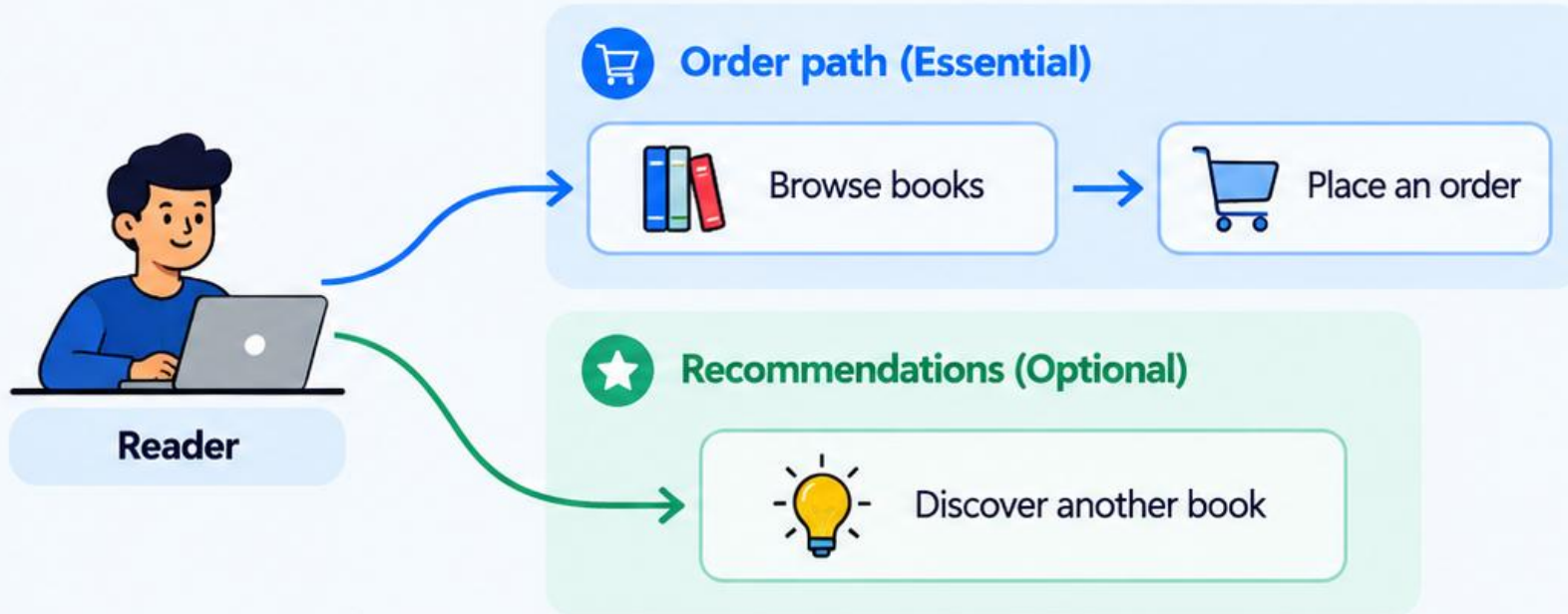
Follow these steps to design a microservices-based BookStore that is **scalable, reliable and resilient** for real-world users.

Understand the problem before choosing the architecture.

BookStore workshop — start with the user journey



User journey in the BookStore



What are we designing?



Browse books



Place an order



Discover another book



The key question

What must still work when recommendations fail?

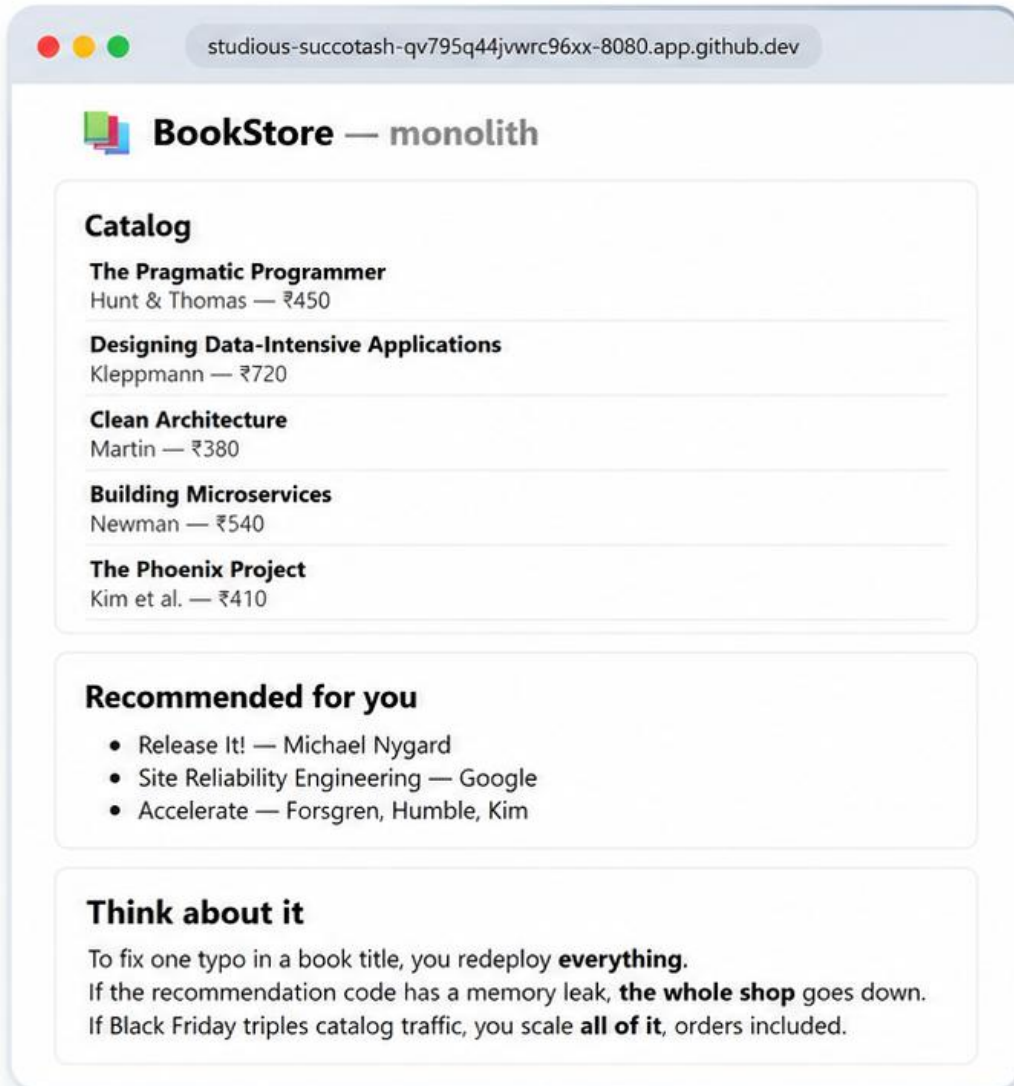


Design exercise

Which activity is essential to the order? Which feature can fail without blocking it?

BookStore – Monolith

One application, all features in a single deployable.



Monolith Architecture



Key Points

- All features (catalog, orders, recommendations) in one application.
- Single codebase, single deployment.
- No network calls between services.
- Simple to build and run.

When it works well

- Small application / team
- Simple requirements
- Early stages or prototypes

Limitations

- Hard to scale parts independently
- One failure can bring down the whole app
- Slower development as the codebase grows
- Requires redeploy for every change

What Are We Designing?

BookStore — from requirements to a justified design



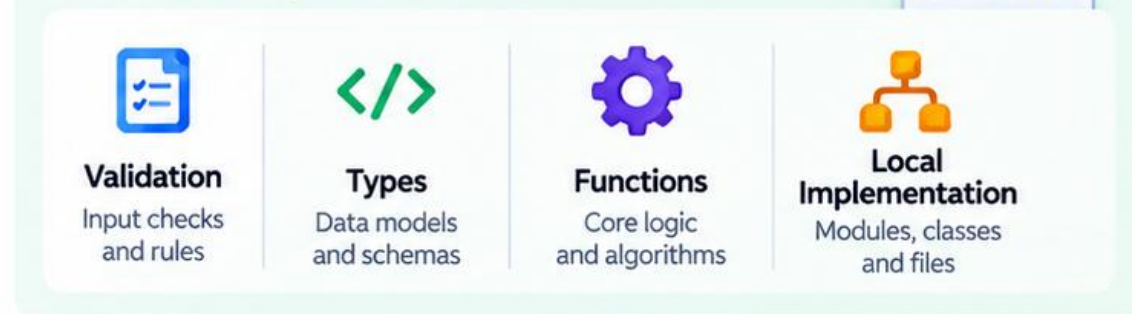
High-Level Design (HLD)

Overall system design and architecture



Low-Level Design (LLD)

Detailed design of each service



A good design is a set of **justified decisions** — not a list of technologies.

Make Requirements Testable

Good requirements tell us what success looks like — and how we prove it.



Requirement

How would we prove it?



Recommendations can fail without blocking an order
Recommendations are optional.

Simulate recommendation service failure → order still succeeds.



Catalog p95 < 300 ms
Catalog should respond quickly under expected load.

Measure API latency (p95) under a stated load (e.g., 100 rps).



One checkout intent = one order
Retries or duplicate requests should not create multiple orders.

Send concurrent retries for the same checkout intent → verify only one order is created.

Today's scope



No real payment
Use a mock payment or simulate the flow.



No durable production data
Use temporary data for the workshop.



No authenticated users
Assume a single user or anonymous access.



If we cannot test it, it is **not yet a useful requirement.**



Estimate Before Scaling

Rough estimates help us choose the right architecture and set initial capacity.



Assumptions



100,000

daily active users (DAU)



20 catalog API calls

per user per day



1% users place an order

(1,000 orders/day)



Peak: 20% of DAU

active at the same time
(20,000 users)

Estimates (Back of the Envelope)



Catalog traffic

$100,000 \times 20 = 2,000,000$ calls/day
 $= 2\text{M} / 86,400 \approx 23$ calls/second (avg)



Order traffic

1,000 orders/day
 $= 0.01$ orders/second (avg)



Peak traffic

20,000 active users $\times$ 20 calls
 $= 400,000$ calls/hour ≈ 111 calls/second



Data storage (rough)

Assume 1 KB per catalog item
 $\approx 500,000$ items $= \sim 500$ MB (catalog)

Results (Initial Load)



Average load

~ 23 API calls/second



Peak load

$\sim 110\text{--}120$ calls/second
(consider 2–3 $\times$ buffer)



Catalog size

~ 500 MB (initial)
(will grow over time)



These are **rough estimates** to start with.

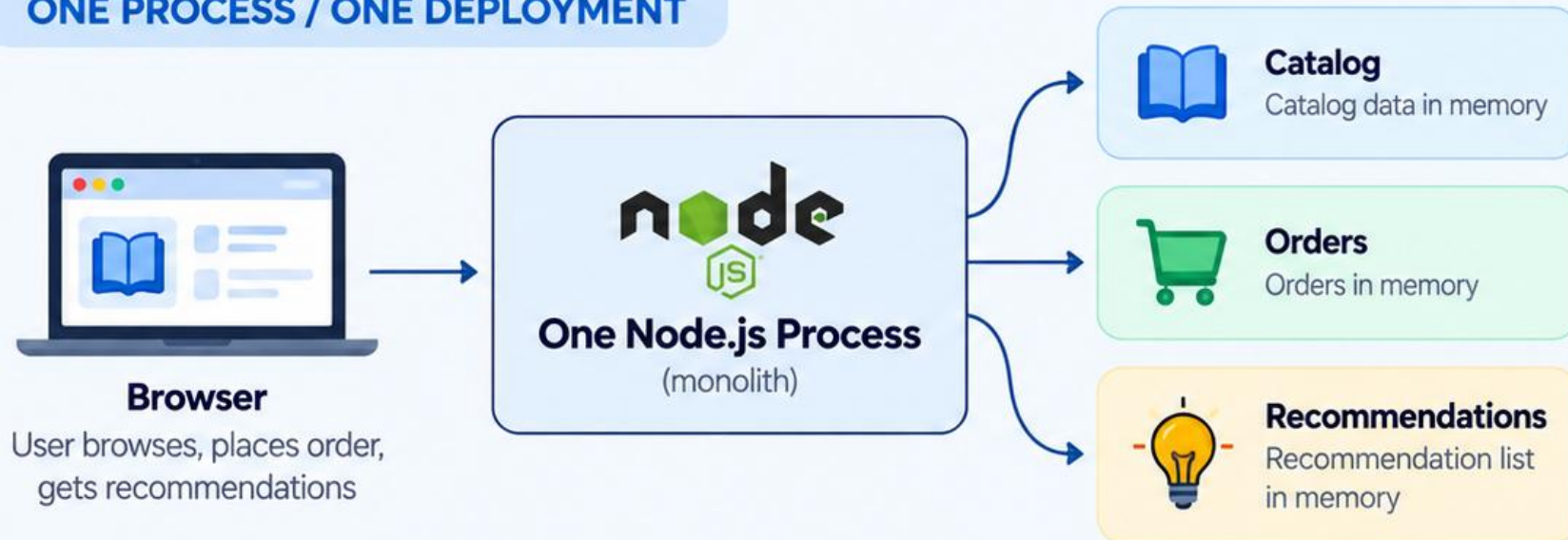
As the product grows, we refine these numbers with real metrics.

Run the Simplest Version First

BookStore — start with one process before adding infrastructure.



ONE PROCESS / ONE DEPLOYMENT



Why start here?



Validate the API

Ensure core functionality works end-to-end.



Observe real behavior

See actual requests, latency and data flow.



Avoid premature infrastructure

Keep it simple and focus on the problem first.



Hands-on

Run the app and test a few APIs

1. Start the application

```
$ node monolith/server.js
```

2. Get books from catalog

```
$ curl http://localhost:8080/api/books
```

3. Place an order

```
$ curl -X POST http://localhost:8080/api/orders \
-H "Content-Type: application/json" \
-d '{"bookId": 1, "title": "The Pragmatic Programmer"}
```

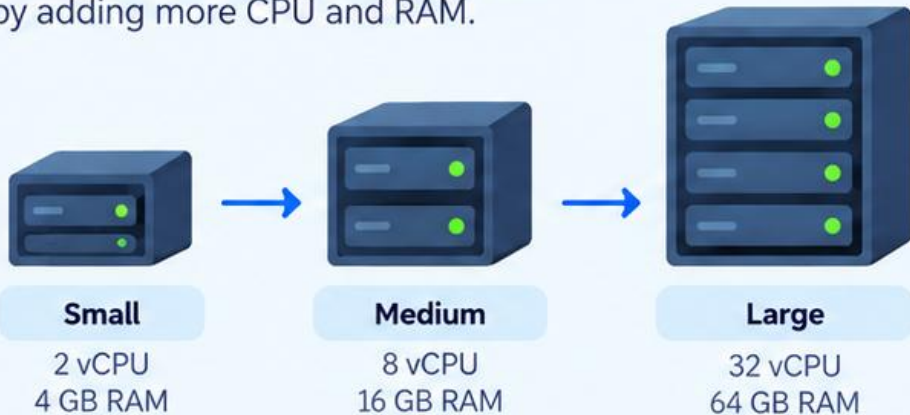
i Data is in memory — intentionally simple for the first step.

A Monolith Can Scale

A single deployable application can also handle large traffic — in two ways.

VERTICAL SCALING

Make the single instance more powerful by adding more CPU and RAM.

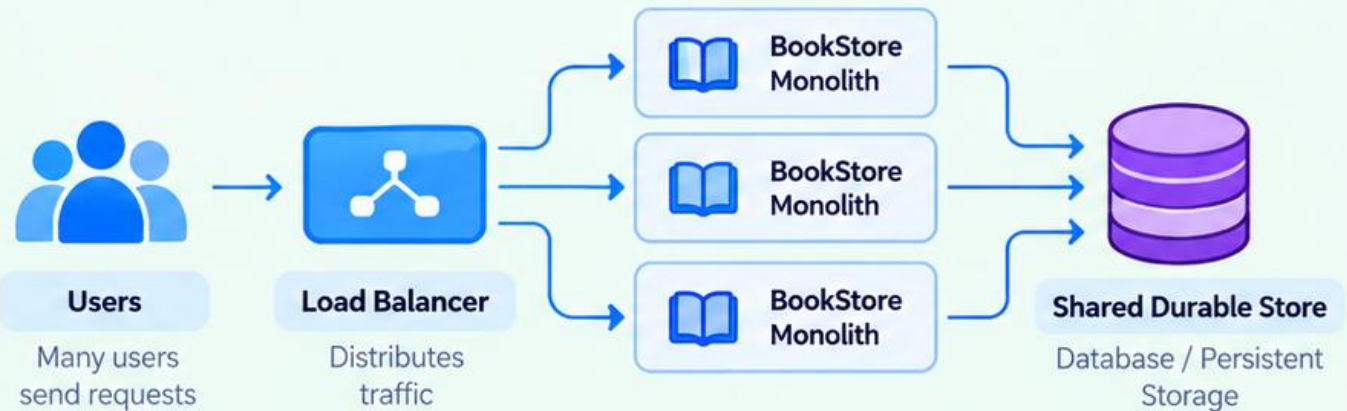


More CPU / RAM

Handle more users and requests

HORIZONTAL SCALING

Run multiple identical instances behind a load balancer.



Vertical = bigger machine

Increase CPU, RAM, disk of the same instance.



Horizontal = more instances

Run multiple copies of the same application behind a load balancer.



More copies **do NOT fix a database bottleneck.**

If the database is the bottleneck, adding more application instances will not solve it.



First find the **real bottleneck**: CPU, memory, I/O, database, or state.

Try a Modular Monolith First

Keep one deployable, but structure the code into clear modules.



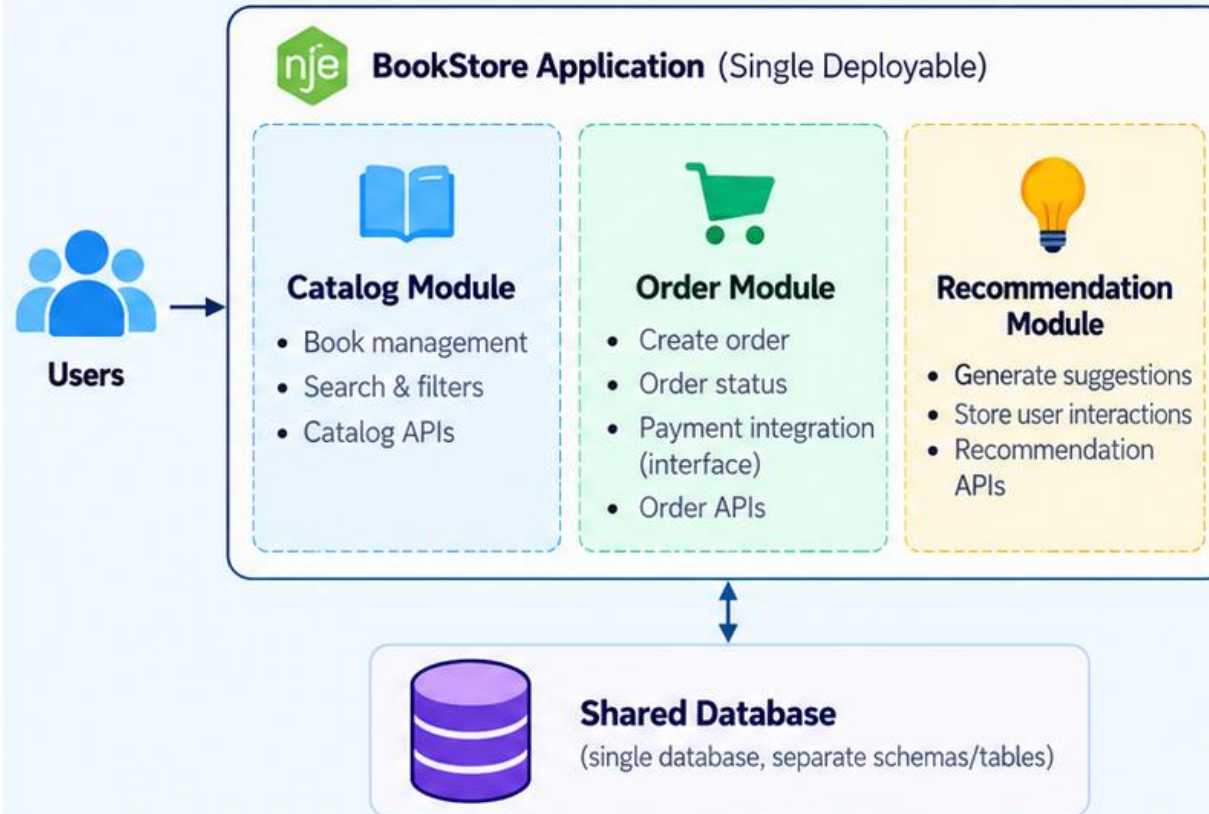
What is a Modular Monolith?

- One deployable application and database.
- Code is organized into separate modules with clear boundaries.
- Each module has its own interfaces and data model.
- Easier to maintain, test and evolve to microservices later.

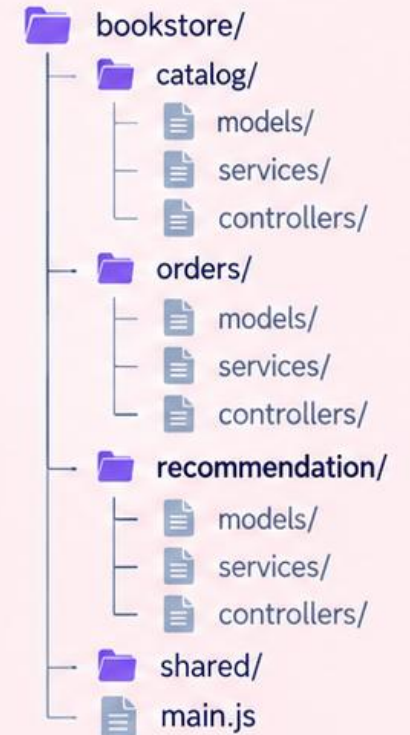
Benefits

- ✓ Clean structure and separation of concerns
- ✓ Independent development
- ✓ Gradual migration to microservices
- ✓ Still simple to deploy and operate

Modular Monolith Architecture (BookStore)



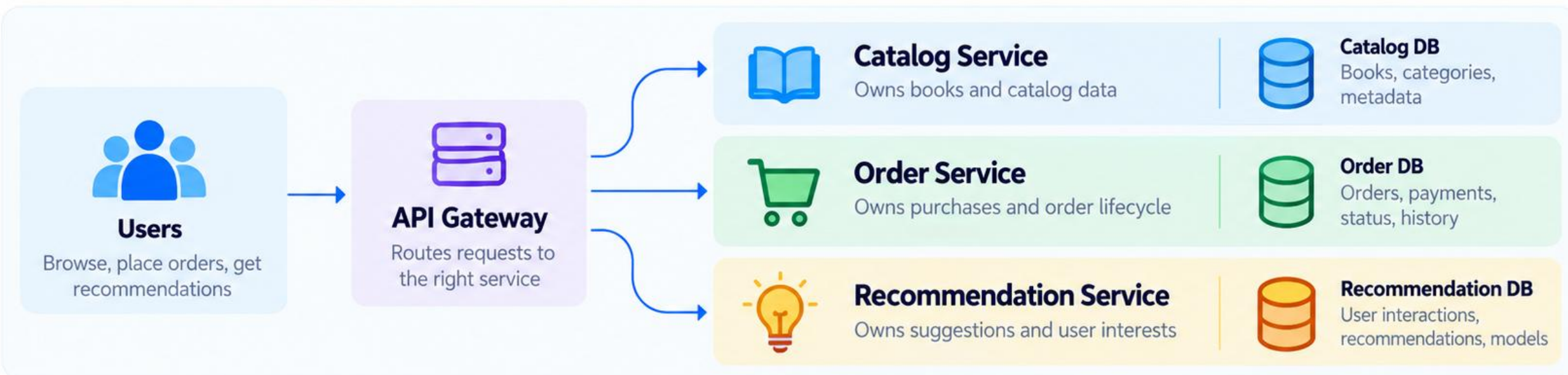
Possible inner structure



A modular monolith gives us **structure today** and **flexibility for tomorrow**.

Boundaries Follow the Business

A service boundary should reflect a real business responsibility.



Boundary Tests

Use these questions to identify a good service boundary.



Responsibility

Can the team explain what it owns?



Change

Can it change without coordinated releases?



Data

Who controls the data and rules?



Failure

What remains useful when this service is unavailable?



Good boundaries = clear ownership + independent change.

Defend Your Boundaries

Don't split by technology — split by responsibility.



College Portal Capabilities

-  **Registration**
Student sign-up and profile
-  **Courses**
Browse and enroll in courses
-  **Fees**
View and pay fees
-  **Attendance**
Mark and view attendance
-  **Results**
View exam results
-  **Library Lending**
Search and borrow books
-  **Hostel Rooms**
Apply and manage rooms
-  **Alumni Newsletter**
Send updates to alumni

Example Business Boundaries

One possible way to group responsibilities:

-  **Academic**
Manages learning and student progress
 - Courses
 - Attendance
 - Results
-  **Student Services**
Manages student lifecycle and payments
 - Registration
 - Fees
-  **Library**
Manages book inventory and lending
 - Library Lending
-  **Hostel**
Manages accommodation and room allocation
 - Hostel Rooms
-  **Alumni / Communications**
Manages alumni engagement and newsletters
 - Alumni Newsletter
-  **Multiple valid boundaries can exist.**
This is one example, not the only correct answer.

Your task

- 1 Propose 3–5 responsibilities**
Group the capabilities in a sensible way.
- 2 State who owns each one**
Which team is responsible?
- 3 Defend why each should be separate or together**
Use business needs, change, data and failure as your reasoning.

Responsibility	Owns	Needs from others	Why separate / together?



A boundary is good when **ownership is clear** and change can happen with **less coordination**.

Independence Has a Bill

Microservices are independently deployable — but they require more discipline.



What does independence mean?

- Teams can deploy, scale, and release services independently.
- Each service owns its data, configuration and operational decisions.
- Services can evolve, be upgraded or replaced without coordinating the whole system — but this introduces new responsibilities.

In a microservices world



New responsibilities

- Coordinate shared changes (e.g., API contracts)
- Distributed monitoring and diagnostics
- Handle data consistency and failures
- Manage dependencies between services
- More operational overhead (deployment, scaling, observability, cost)

Need		Possible benefit	New responsibility	Example
	Separate catalog capability	Move catalog faster	Shared library or API contracts needed	Catalog service evolves book data model
	Optional recommendations	Enable A/B testing and experimentation	Data pipelines and feature sync	Recommendation service uses catalog events
	Independent releases	Less risk, faster iterations	API compatibility and integration tests	Order service released without changing catalog
	Own data	Autonomy and clear ownership	No shared tables, handle consistency	Each service has its own database



Independence gives flexibility, but it comes with a cost.

Design for it: contracts, observability, data ownership, and operational discipline.

First Checkpoint

Turn requirements into a simple, modular monolith with clear verticals.



Goal

Structure the code around business verticals before scaling or splitting into services.

1. Understand Requirements



Identify users and key flows

e.g., student registration, course enrollment, fee payment, etc.



List business capabilities

e.g., registration, courses, fees, attendance, library, hostel, alumni



Find common and shared needs

e.g., authentication, notifications



Note constraints

e.g., single deployment, shared DB (for now)



2. Define Vertical Modules

Group related features into modules (verticals).



College Portal (Modular Monolith)



Registration Module

- Student sign-up
- Profile management
- Authentication



Course Module

- Browse courses
- Enroll in courses
- Course details



Fees Module

- View fees
- Make payment
- Payment history



Attendance Module

- Mark attendance
- View attendance
- Reports



Library Module

- Search books
- Borrow/return
- Manage inventory



Hostel Module

- Apply for rooms
- Room allocation
- Track requests



Shared / Common Modules (e.g. Auth, Notifications, Audit, Config)



3. Implement Vertical Strategies

Inside one codebase, keep each module separate.



Separate folders

src/registration/, src/courses/, ...



Clear interfaces

Define module boundaries (services, repositories, controllers)



Own data model per module

Separate models/tables (or schemas)



Minimize cross-module dependencies

Use well-defined interfaces/events



Common modules for shared needs

Auth, notifications, logging, config



Result



A modular monolith that is easy to understand



Clear module boundaries aligned with business capabilities



Ready to evolve into microservices in the future

One Public Entry Point

All client requests go through an API Gateway, which routes to the right service.

Key Responsibilities



Route requests

Send each request to the right service (e.g., /catalog, /orders)



Enforce policies

Authentication, authorization, rate limits, and quotas



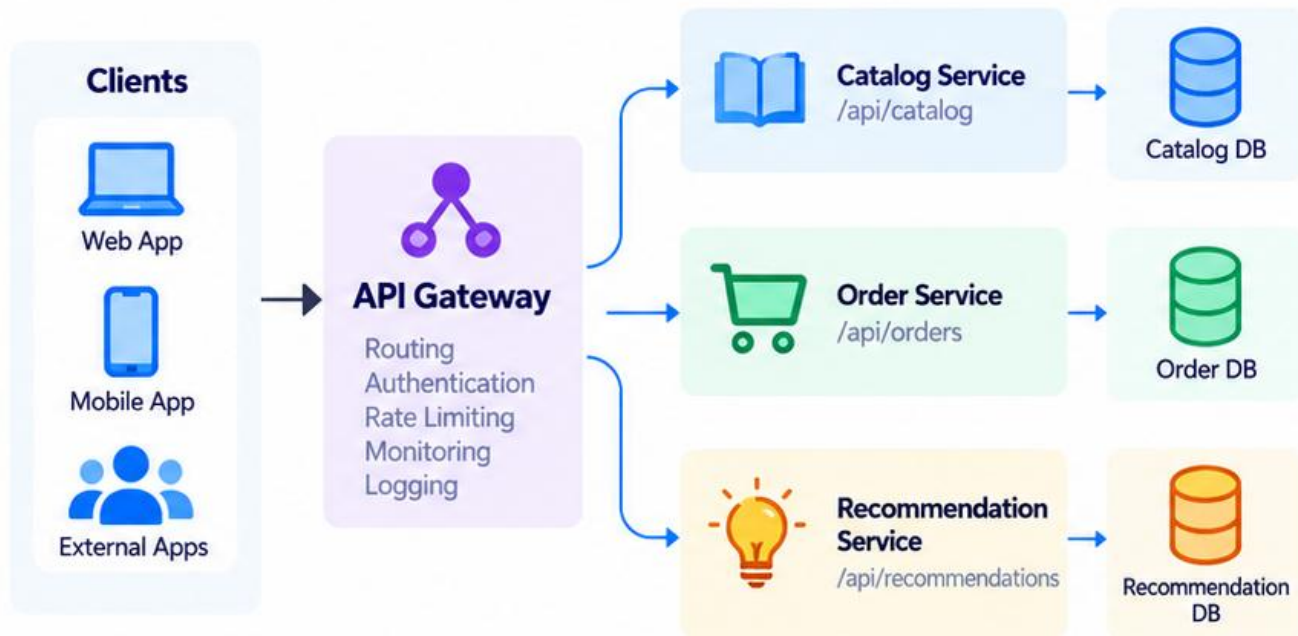
Keep business logic in services

Gateway only handles routing and cross-cutting concerns



Protect and scale

Handle traffic spikes, failures, and scaling of backend services



Benefits



Security

Centralized auth, rate limits, and policies



Simpler clients

One entry point for all APIs



Observability

Centralized logging, monitoring, tracing



Scalability

Scale gateway and services independently



Loose coupling

Services can evolve without changing clients



Important Notes

- The gateway is NOT a place for business logic like "order processing" or "book recommendations".
- No need for a publicly accessible database or internal service ports.
- The gateway can be a single or multi-instance component (it is a potential failure point, so run multiple instances).
- Authentication, rate limiting, and request validation are common responsibilities.
- Order logic, catalog logic, and recommendation logic remain in their own services and databases.

Find By Name, Route To Instances

Use the service name to find available instances and let the gateway route the request.

1

Call the service by name

```
http://catalog:3001/...
```

Use the service name (catalog) and its port.

2

Discover available instances

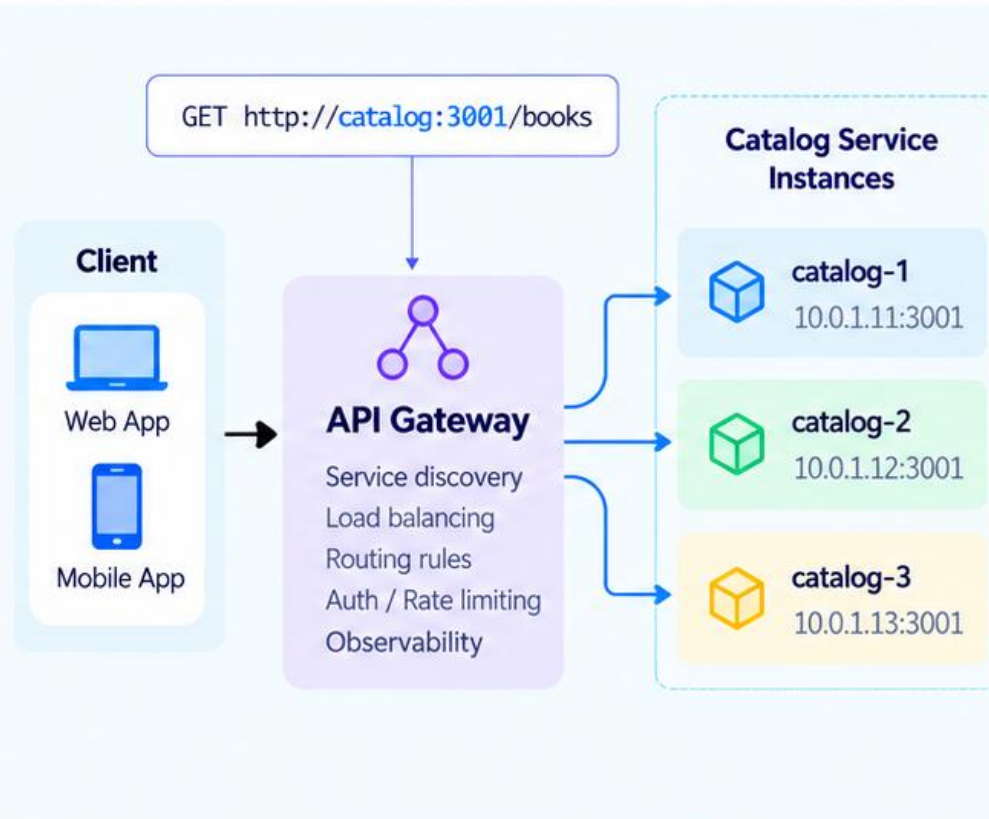
The platform (e.g., Docker/Kubernetes) resolves the name to multiple running instances.

Service: catalog (port 3001)

catalog-1	10.0.1.11	:3001
catalog-2	10.0.1.12	:3001
catalog-3	10.0.1.13	:3001

Example request

```
curl -i http://catalog:3001/books
```



3

Load balance and route

The gateway routes the request to one of the healthy instances.



Routing strategies

- Round Robin (default)
- Least Connections
- Weighted / Custom Rules
- Health checks

4

Service handles the request

The selected instance processes the request and returns a response.



Example response

```
{
  "bookId": 1,
  "title": "The Pragmatic Programmer"
}
```



Key Takeaways

- Use the service name, not individual container IPs.
- The platform resolves the name to multiple instances.
- The gateway handles routing, load balancing, and failures.
- Your application code does not need to know instance details.

Trace A Real Request

Follow a request from the browser to the actual service instance.

1

Send a request to catalog

Use the public API endpoint.

```
curl -i http://localhost:8080/api/books
```

2

Gateway routes the request

/api/books is routed to the catalog service.

3

Catalog service processes

Returns the books JSON response.

4

Find the serving instance

Check which container/pod handled it.

>_

Run the lab

```
# Start services
docker compose up --build -d

# Check running containers
docker compose ps

# Call the API through gateway
curl -i http://localhost:8080/api/books

# (Optional) check logs
docker compose logs -f gateway catalog
```



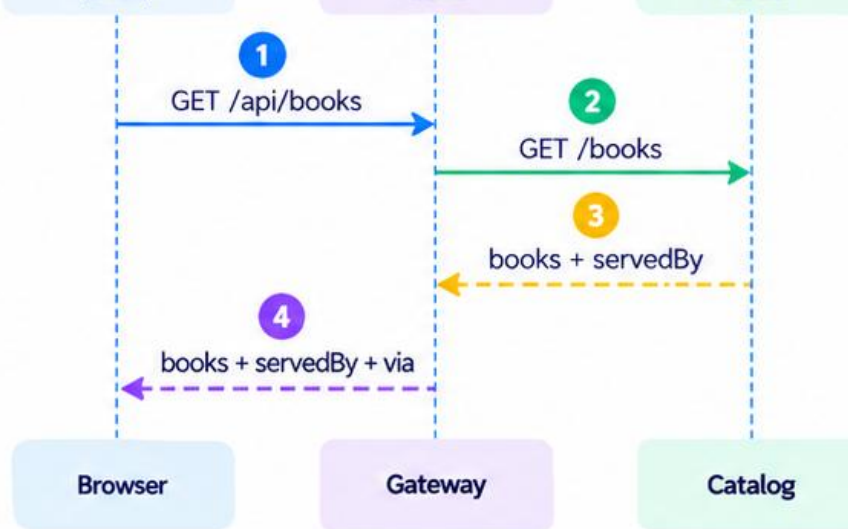
Browser
(Client)



API Gateway
:8080



Catalog Service
:3001



5

Inspect the response



Inspect the JSON response

The response includes which instance served it.

```
{
  "books": [
    { "id": 1, "title": "The Pragmatic Programmer" }
  ],
  "servedBy": "catalog-1",
  "instance": "10.0.1.11:3001"
}
```

6

Verify the running instance

Use docker or kubectl to see the instance.

Docker (Compose)

Kubernetes

docker compose ps

NAME	IMAGE	PORTS
catalog-1	catalog:latest	0.0.0.0:3001->3001/tcp
catalog-2	catalog:latest	0.0.0.0:3002->3001/tcp
catalog-3	catalog:latest	0.0.0.0:3003->3001/tcp



Files involved

docker-compose.yml

```
services:
  gateway:
    ports:
      - "8080:8080"
  catalog:
    ports:
      - "3001:3001"
```

gateway.yml (routing)

```
routes:
  - path: /api/books
    service: catalog
    url: http://catalog:3001
```



What to look for

- The response JSON contains servedBy or instance.
- The gateway only routes, it does not contain business logic.
- Multiple catalog instances can serve the same request.
- Try multiple requests and see different instances (load balancing).
- You can simulate a failure by stopping one catalog instance and retrying.



Key Takeaway: The gateway provides a stable entry point, routes to healthy service instances, and hides internal details from clients.

Contracts and Ownership

Well defined API contracts and clear ownership enable independent development.



API contracts

Define request/response format, validation rules and versioning.



Service ownership

Each service owns its code, data (database) and API.



Independent evolution

Services can change and deploy without blocking others as long as the contract is maintained.

API Calls and Ownership



Public request	Actual internal call	Classroom limitation	Owner	Notes
GET /api/catalog	GET /catalog:3001/books	Load inventory, catalog	Catalog team	Catalog owns code and data. Others use this API, not the database.
POST /api/orders	POST /orders:3002/orders	Create order, deduct stock	Orders team	Orders owns ordering logic and data.
GET /api/recommendations	GET /recommendations:3003/recommendations	Suggest books	Recommendations team	Reco owns model and data. Can evolve independently.



Key Takeaways

- Define clear API contracts (schema, validation, versioning).
- Each service owns its code, database and business logic.
- Use the API gateway to communicate across services, not direct database access.
- Independent deployments are possible if the contract is respected.

Sync or Async?

Choose based on what the caller needs to know.



Synchronous

Used when the caller needs the result immediately.



Need the result now

e.g., Fetch books to show on screen



Request → Response

Direct API call between services



Caller waits

The next step happens after the response



Asynchronous

Used when the result can happen later.



Result can happen later

e.g., Place order and send confirmation, update recommendations



Event / Queue

Publish an event (e.g., "OrderPlaced") to a message broker



Caller continues

Does not wait for other services to process the event



Need an immediate answer? → Sync

Example: Fetch books from catalog

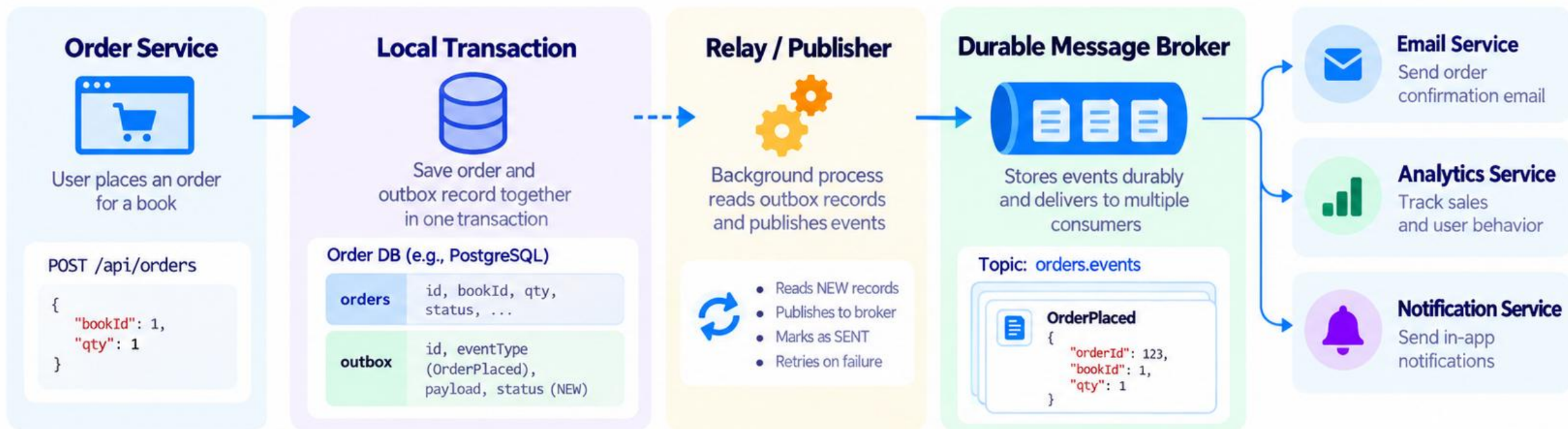


Announcing something that happened? → Async

Example: Place order, send confirmation, update recommendations

A Log Is Not A Message Broker

Writing an event to a log is not the same as reliably delivering it to consumers.



Log

= record of what happened

- Persists events for debugging, auditing and history.
- Does not deliver events to consumers.
- Consumers cannot rely on logs for real-time processing.
- Example: outbox table, application logs, audit logs.



Outbox

= closes the DB-write + event-publish gap

- Order and event (e.g., OrderPlaced) are saved together in one database transaction.
- A separate relay process publishes outbox records.
- Ensures no events are lost even if the service crashes.
- Marks records as **SENT** after successful publish.



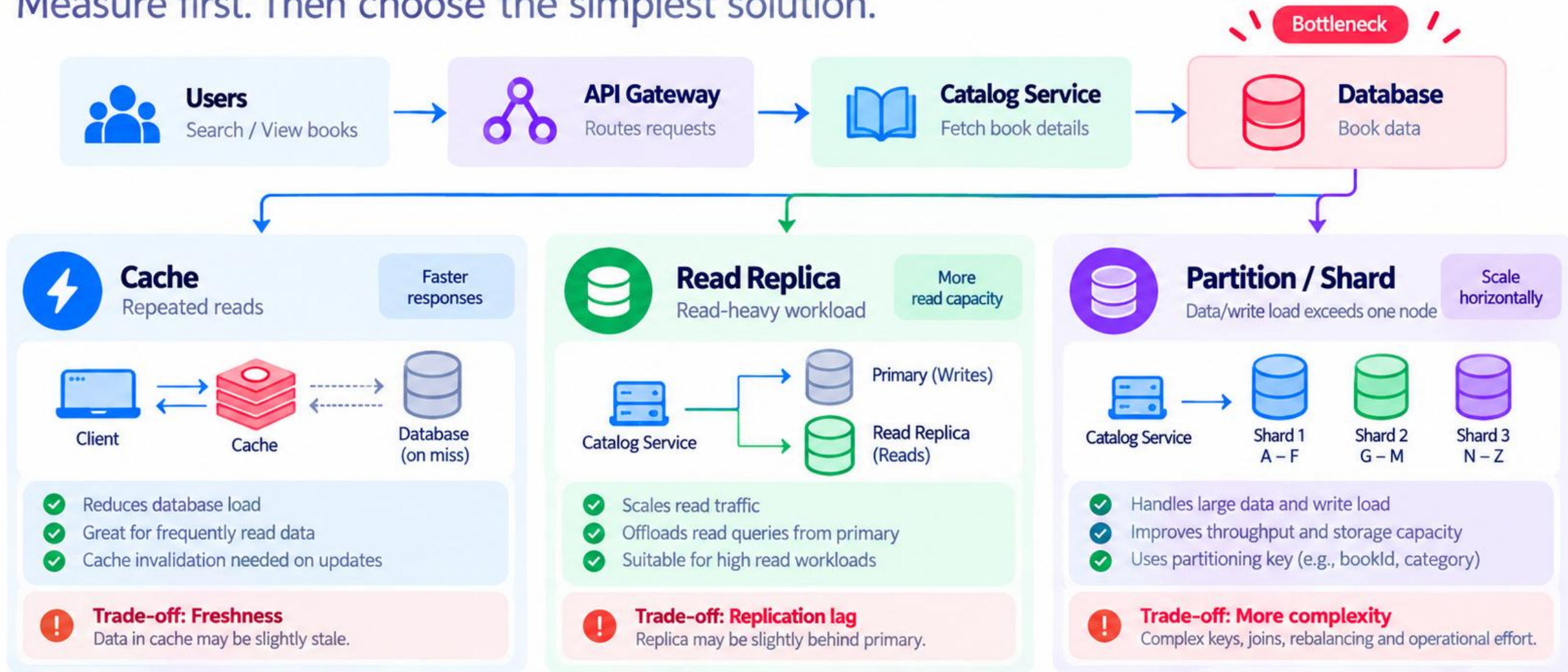
Broker

= deliver events to consumers

- Stores events durably.
- Delivers to multiple independent consumers.
- Handles retries, backpressure and failure management.
- Consumers process events at their own pace.
- Example: Kafka, RabbitMQ, AWS SQS/SNS, Pulsar.

Scale the Bottleneck, Not the Diagram

Measure first. Then choose the simplest solution.



Key Takeaway: Don't add infrastructure because the diagram looks empty — **add it because measurements show a bottleneck.**

Break A Dependency, Observe The Decision

Systems should handle failures gracefully with timeouts, retries and graceful degradation.



1 System running normally

- Recommendations are shown within ~200ms
- All services healthy

2 Stop the dependency

- Stop the recommendations service or simulate slow response
- Catalog continues to work

3 Observe behavior

- Recommendations fail/timeout after few seconds
- Catalog still responds
- Application does not crash

4 Check logs and metrics

- Gateway shows timeout errors
- Caller logs show dependency exhausted / retries
- Notice increased latency and error rate

5 Restore the environment

- Start the recommendations service
- Verify system recovers automatically

>_ Simulate the failure

```
# Stop recommendations service
docker compose stop recommendations

# Or add delay / slow responses
docker compose exec recommendations \
  tc qdisc add dev eth0 root netem delay 5000ms'
```

🔍 What to observe?

```
[Gateway] ERROR recommendations timeout after 2000ms
[Caller] WARN recommendations falling back
[Catalog] INFO /api/books 200 OK (still works)

[Gateway] ERROR recommendations connection exhausted
[Caller] WARN retry attempts exhausted
[App] INFO graceful degradation enabled
```

📊 Decision

- ✓ Recommendations dependency failure should not bring down the whole system.
- ✓ Return partial results (e.g., books only).
- ✓ Use timeouts, retries with backoff and circuit breaker.
- ✓ Monitor and alert on dependency health.
- ✓ Restore the service and verify automatic recovery.

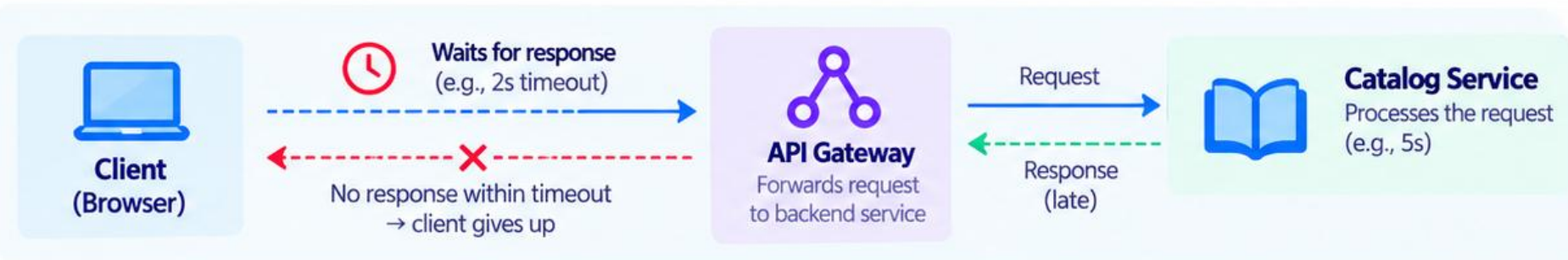


Key Takeaways

- Always define timeouts and retries for external calls.
- Use graceful degradation and return partial results.
- Observe system behavior using logs, metrics and traces.
- Validate recovery and ensure the system is resilient to cascading failures.

Timeout Does Not Mean Nothing Happened

A timeout only means the client stopped waiting for a response.



Key Point

The client timed out, but the service may still be processing and successfully completed the request later.

1 Client sees a timeout

```
HTTP/1.1 504 Gateway Timeout
Content-Type: application/json
```

```
{
  "error": "Request timed out",
  "message": "Please try again"
}
```

2 Check backend logs

```
2026-09-23 10:30:12 INFO Received request
2026-09-23 10:30:12 INFO Processing request ...
2026-09-23 10:30:17 INFO Completed successfully
2026-09-23 10:30:17 INFO Response sent (after 5s)
```

3 Database / Service state

```
{
  "orderId": "12345",
  "status": "COMPLETED",
  "createdAt": "2026-09-23T10:30:12Z",
  "updatedAt": "2026-09-23T10:30:17Z"
}
```



What happened?

- Client timed out after waiting (e.g., 2 seconds).
- Backend service continued processing the request.
- The operation completed successfully after 5 seconds.
- Data is persisted, even though the client received a timeout.
- If the client retries, it may create a duplicate unless the operation is idempotent.

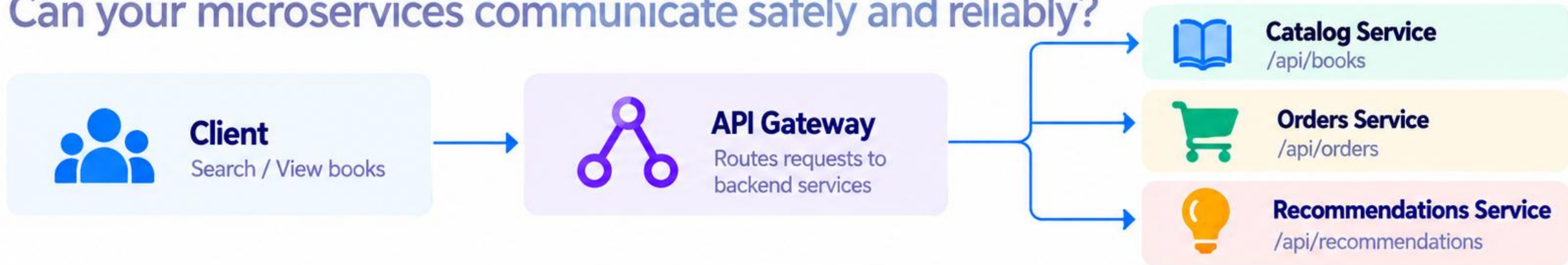


Key Takeaways

- 1 Timeout only means the client stopped waiting, not that nothing happened.
- 2 Always check backend logs and data state.
- 3 Use idempotency keys to avoid duplicates on retries.
- 4 Configure appropriate timeouts and monitor long-running requests.
- 5 Communicate clear retry guidance to clients.

Second Checkpoint

Can your microservices communicate safely and reliably?



Resilience Layer



Timeout

Don't wait forever



Retry

Handle transient failures



Fallback

Graceful degradation



Idempotency

Avoid duplicate operations

1

Where does a request enter?



Client calls a single entry point (API Gateway).

2

How does a service find another service?



Service discovery (using a catalog like Kubernetes/Consul/ECS).

3

When should we use sync vs async?



Sync for immediate responses, async for long-running tasks (using events/queues).

4

What happens when a dependency fails?



Use timeouts, retries, fall back to cached/default response, and keep the system stable.

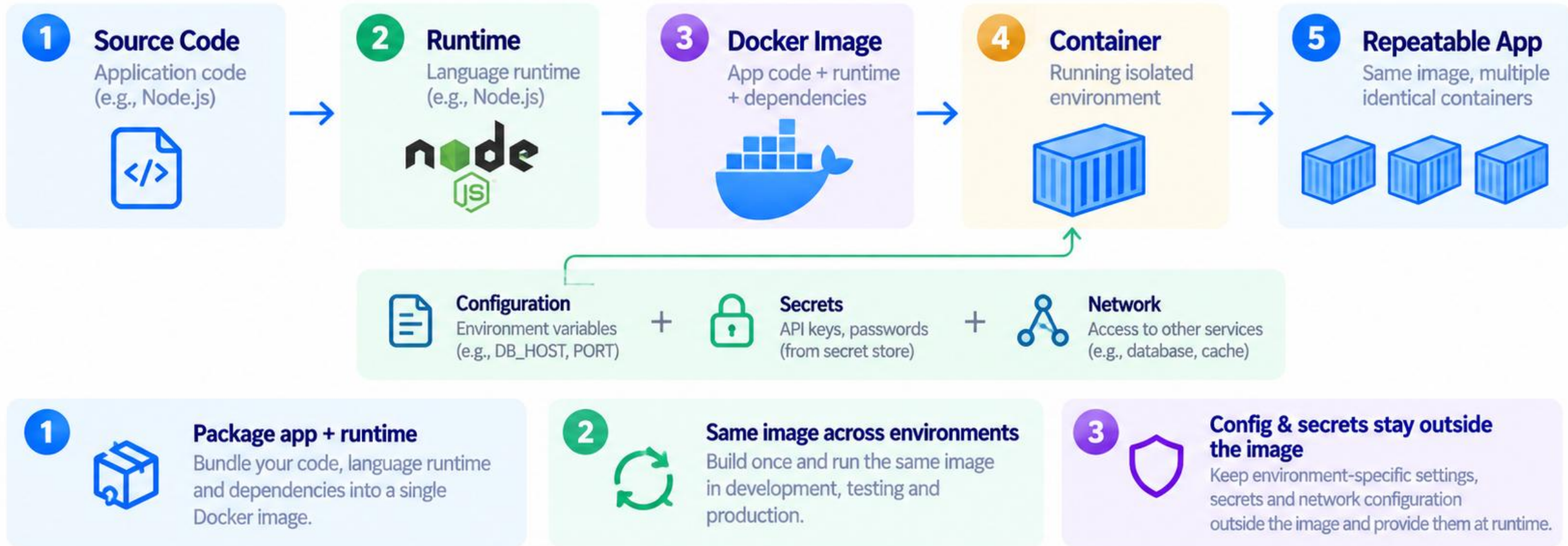


Key Takeaway: Good architecture is not just about connecting services — it is about **handling failures gracefully**.

Session 3: Containerization & Orchestration

From Code to a Repeatable Runtime

Package your application and its runtime into a container, and run it the same way everywhere.



Key Takeaway: Build once → Run consistently.

A container gives you a repeatable, portable and isolated runtime for your application.

Containers Are Not Tiny VMs

Containers share the host OS kernel, making them lightweight and fast.

1 What containers are

- Containers package an application with its runtime, libraries and configuration.
- They run as isolated processes on the same host OS kernel.
- Each container has its own filesystem, network and resource limits.



2 What containers are NOT

- Not a full operating system (no separate kernel).
- Not a virtual machine.
- Not heavy or slow like VMs.
- Not meant to run multiple kernels or full OS per container.



Aspect	 VM (Virtual Machine)	 Container
 OS / Kernel	Each VM has its own guest OS and kernel	Shares the host OS kernel
 Size	Large (GBs)	Small (MBs)
 Startup time	Slow (tens of seconds to minutes)	Fast (milliseconds to seconds)
 Resource usage	Higher CPU, memory and storage	Lower resource usage
 Isolation	Stronger (hardware virtualization)	Process-level isolation (namespaces, cgroups)
 Use case	Run different operating systems	Run applications consistently across environments



Key Takeaway: Containers are lightweight, fast and portable — they **share the host OS kernel**, not a full OS like VMs.

Recipe, Artifact, Instance

From code to a running container



Key Idea

A Dockerfile is a recipe to build an image (artifact), which can be run as one or more container instances.

1 Recipe (Dockerfile)

Instructions to build the image.



```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "app.js"]
```

2 Artifact (Image)

A read-only image with app, runtime and dependencies.



Image contains:

- Application code
- Runtime (Node.js)
- Dependencies (node_modules)
- Configuration files

3 Instance (Container)

A running container created from the image.



Container is a running process of the image:

- Isolated environment
- Uses host OS kernel
- Own filesystem (from image)
- Can run multiple instances

4 Workflow

Source Code
(app files)



Dockerfile
(Recipe)

docker build

Image
(Artifact)

docker run

Container Instances
(Running app)

! Important

- An image is immutable (read-only).
- Containers are lightweight, isolated processes.
- You can run many container instances from the same image.



Lab connection

Run `docker build -t bookstore:v1 .` and inspect the image using `dacker images`.
Then run the container using `dacker compose` and identify the `bookstore:v1` image.

Build And Inspect Catalog

Build a Docker image and see how it appears in the local catalog (image registry).

1

Dockerfile

Defines how to build the image.



<> Dockerfile

2

Build image

Creates an image from the Dockerfile.



```
docker build -t bookstore:v1 .
```

3

Image in catalog

Image is stored locally in Docker image catalog.



docker images

4

Run / Publish

Run containers or push to a registry later.



Example Dockerfile

```
1 FROM node:22-alpine
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 EXPOSE 3000
7 CMD ["node", "server.js"]
```

Build the image

```
# Build the image and tag it
docker build -t bookstore:v1 .
Generating build context...
Step 1/7 : FROM node:22-alpine
....
Successfully built abc123def456
Successfully tagged bookstore:v1
```

Inspect the local catalog

```
# List images in local Docker catalog
```

```
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
bookstore	v1	abc123def456	2 minutes ago	186MB
node	22-alpine	d34db33f1234	3 days ago	186MB
nginx	latest	9f8e7d6c5b4a	2 weeks ago	142MB



Key Takeaways

- Dockerfile is a recipe to build an image.
- An image is an artifact (immutable).
- `docker images` shows all local images.
- Tags (e.g., v1) are just pointers to an image ID.

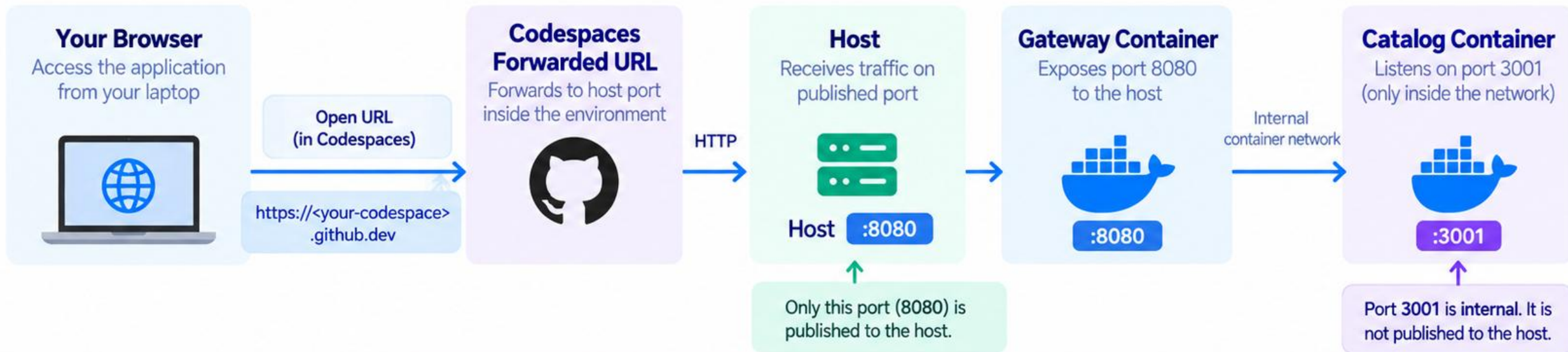


Lab Connection

- 1 Run `docker build -t bookstore:v1 .`
- 2 Run `docker images` and identify the `bookstore:v1` image.
- 3 Understand the columns: REPOSITORY, TAG, IMAGE ID, CREATED, SIZE.
- 4 (Optional) Run a container: `docker run -p 3000:3000 bookstore:v1`

Which Port Is Where?

Understand host ports, container ports, and forwarded URLs.



Container port

- Where the application listens inside the container.
- E.g., catalog listens on `3001`.



Host port

- Published entry point from host to a container.
- Only port `8080` is published in this setup.



localhost

- The environment where you run the command.
- Inside Codespaces, `localhost` refers to the host (VM), not your laptop.



Codespaces

- Forwards a host port to your browser via a public URL.
- You access the app using the forwarded URL (not `localhost`).



Docker port mapping example

```
docker run -p 8080:8080 gateway
```

↑ ↑
Host port Container port
(on Codespaces VM) (inside container)

```
docker run -p 3001:3001 catalog
```

↑ ↑
Host port Container port



Lab Discussion

- 1 Explain why `wget` runs inside the gateway container.
- 2 Why does port 3001 appear in Ports only after publishing it with `-p`?

Restart Policy **Versus** Workload Reconciliation

Why your container comes back automatically and how Compose keeps the desired state.

1 Docker restart policy

What happens when a container exits?

no	Default. Do not automatically restart.
always	Always restart, even after manual stop.
on-failure	Restart only if the container exits with a non-zero code.
unless-stopped	Always restart, unless it was manually stopped.

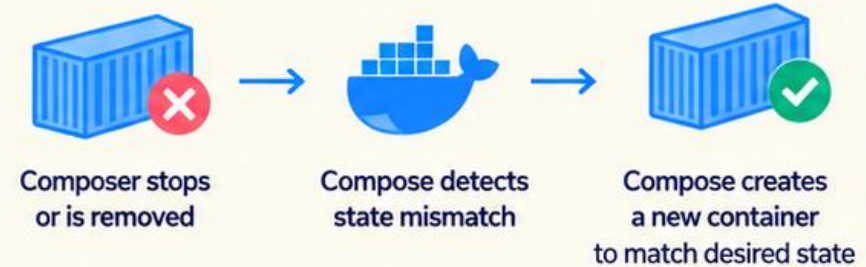
2 Workload reconciliation (Compose)

Keeps the desired state running.

- Compose continuously checks the current state with the desired state (replicas).
- If a container stops or is deleted, Compose creates a new one automatically.
- It maintains the desired number of containers as defined in the configuration.

3 Example flow

Catalog container exits and is restarted.



4 Observed events

Event	Meaning
Catalog container exits here	No configured restart policy to reload state.
Container remains in a Bad state	Kubelet may restart it within the Pod.
Managed Pod is deleted	Controller creates a replacement Pod.

5 Lab: Observe and test

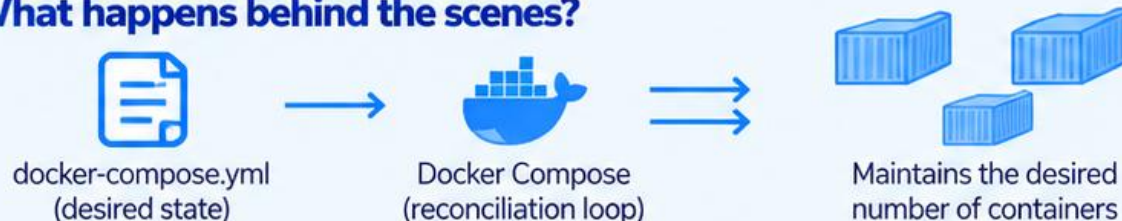
```
# 1. Bring up the catalog container
docker build -t catalog-container:latest .
docker compose up -d

# 2. Check running containers
docker compose ps

# 3. Manually stop the container
docker stop $(docker compose ps -q catalog)

# 4. Observe that Compose creates a new one (reconciliation)
watch docker compose ps
```

6 What happens behind the scenes?



Key Takeaways

- 1 Restart policy defines what Docker does when a container exits.
- 2 Compose/Kubernetes continuously ensures the desired state is running.
- 3 If a container is stopped or deleted, it is recreated automatically.
- 4 Use `docker compose ps` to observe the current state.

Who Keeps The Pods Alive?

Kubernetes continuously works toward the desired state.



Desired state: 3 replicas

Kubernetes ensures the actual state matches the desired state.



Deployment

Manages rollout and version of your application.

```
spec:
  replicas: 3
  template:
    ... (image, labels, etc)
```

creates / manages



ReplicaSet

Maintains the desired number of Pod replicas.

Desired replicas: 3

Pods are scheduled on nodes



Catalog Pod (1/3)



Catalog Pod (2/3)



Catalog Pod (3/3)



Node

A worker machine running your Pods.



Kubelet

Manages containers on this node.

Runs / restarts container

Catalog Pod (on this node)



Container (e.g., bookstore:latest)



Self-healing in action

If a Pod fails, Kubernetes brings it back automatically.



1. Pod fails

Pod crashes or is deleted (e.g., 2/3 running).



2. ReplicaSet detects

Sees actual count (2) < desired count (3).



3. New Pod created

ReplicaSet creates a replacement Pod.



4. 3/3 running

Desired state restored automatically.



Deployment

= rollout & version

- Defines desired state (replicas, image, labels, etc).
- Handles rolling updates and rollbacks.
- Creates and manages a ReplicaSet.



ReplicaSet

= desired Pod count

- Maintains the desired number of Pods.
- Creates new Pods if count is lower.
- Deletes extra Pods if count is higher.
- Uses the Pod template from Deployment.



Kubelet

= runs containers on a node

- Ensures containers in each Pod are running.
- Restarts containers if they exit or crash.
- Works with the container runtime (e.g., containerd).
- Reports Pod status to the control plane.



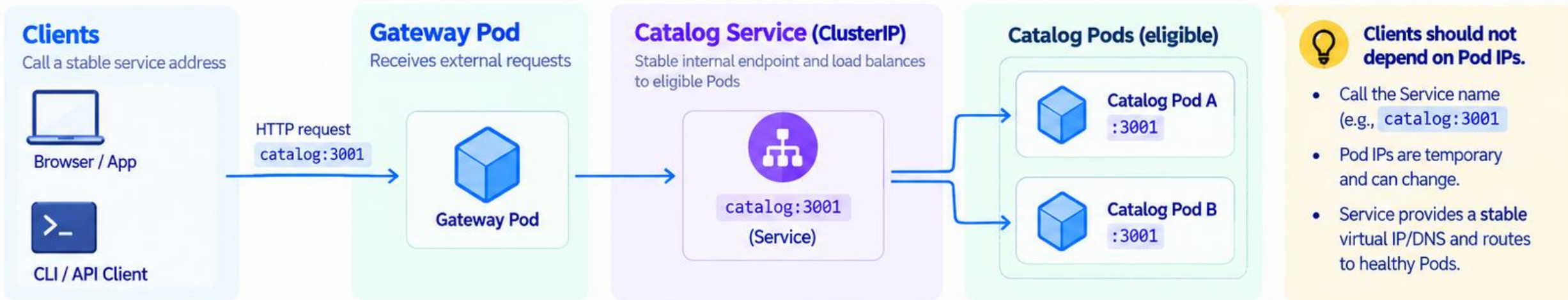
Lab 3C: Inspect bookstore.yaml

Look at the following fields:

- 1 replicas (desired Pod count)
- 2 Pod labels (e.g., app: catalog)
- 3 container image (e.g., bookstore:latest)
- 4 selector (matches the Pods)

A Service Is A Stable Routing Abstraction

Clients call a stable Service name, while the actual Pods can change underneath.



Service = Stable Endpoint

- Has a stable DNS name and **ClusterIP** (e.g., `catalog:3001`).
- Load balances traffic to healthy Pods.
- Clients always call the Service, not individual Pods.



Selector = Finds Eligible Pods

- Service uses a label selector to find matching Pods (e.g., `app: catalog`).
- Automatically routes to all matching, ready Pods.
- No manual updates needed when Pods change.

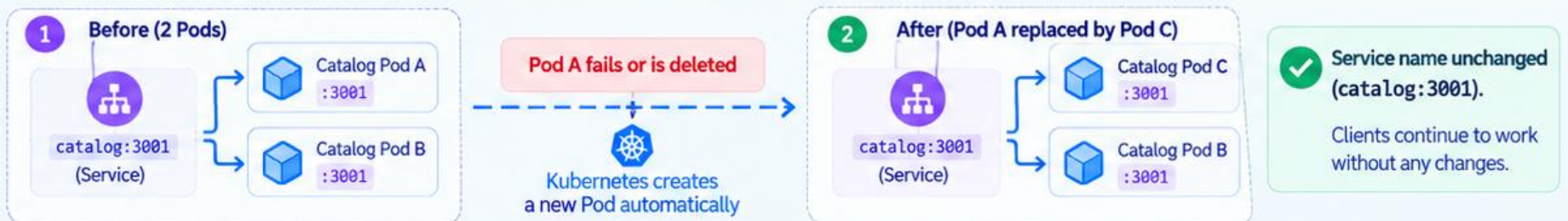


Pods = Replaceable Instances

- Run the application (e.g., on port 3001).
- Can be created, deleted or replaced at any time (e.g., during rollout or on failure).
- Service continues to work as long as there are healthy Pods matching the selector.

Self-healing in action

A Pod can be replaced, but the Service name remains the same.



Running Is Not Ready

A container can be running, but not ready to receive traffic yet.



What each probe does?



Startup Probe

Allows time for the application to initialize. Until this passes, other probes are disabled.



Readiness Probe

Checks if the app is ready to receive traffic. If it fails, the Pod is removed from Service endpoints (but not restarted).

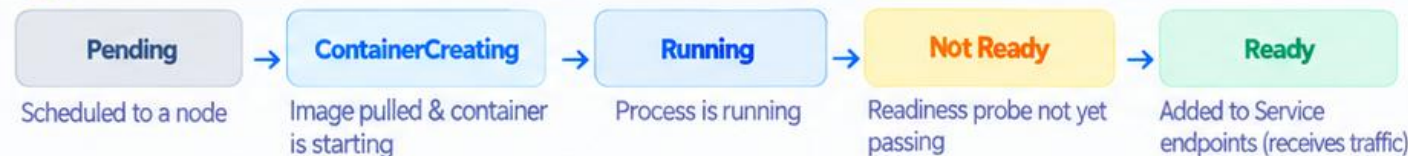


Liveness Probe

Checks if the container is healthy. If it fails, Kubernetes restarts the container.



States during startup



Not all failures restart the container

Readiness probe fails

Pod is removed from Service traffic, but container keeps running.

Liveness probe fails

Kubernetes restarts the container (restarts the process).



Key Takeaways

- A container can be **Running** but not **Ready**.
- **Startup probe** gives time for initialization.
- **Readiness probe** controls traffic (adds/removes Pod from Service).
- **Liveness probe** restarts unhealthy containers.



Lab Task

- 1 Find the `/health` endpoint in the catalog code.
- 2 Confirm there is no `readinessProbe` field in the manifest.
- 3 Identify the missing wiring and discuss how to add a readiness probe.

Configuration And Change Safety

Separate configuration from the image — then control how changes reach production.

External Configuration

Lives outside the container image



ConfigMap

Non-sensitive config
e.g. app environment,
URLs, feature flags



Secret

Sensitive data
e.g. API keys, DB credentials,
tokens (base64/encrypted)

Workload Template

Deployment uses image + external config



Deployment (Pod template)



Container image (immutable)
e.g. bookstore-catalog:v1



Environment from ConfigMap/Secret
envFrom / volumes

New Pods

Pods created with image + config



Pod 1

Image: v1
Config: from CM/Secret



Pod 2

Image: v1
Config: from CM/Secret



Pod 3

Image: v1
Config: from CM/Secret

Traffic to Service

Clients call a stable Service name



Service
(ClusterIP / Ingress)



Web/App



Mobile Client



Other Services

1

Rolling Update

Gradually replace old Pods with new Pods



v1

v1

v1

Old Pods



v1

v2

v2

New Pods (rolling)



Safeguards:

- Readiness probe (send traffic only when ready)
- Spare capacity (keep some old Pods during rollout)
- Connection draining (let in-flight requests finish)

2

Canary Release

Send a small % of traffic to a new version



90% traffic



v1

(Stable)



10% traffic



v2

(Canary)



Monitor metrics

- Error rate
- Latency
- Throughput
- Business metrics



Safeguard:

- Observe metrics before gradually increasing traffic.

3

Blue / Green Deployment

Two identical environments, switch traffic

Blue (Current)



v1

(Live)



Switch traffic

Green (New)



v2

(Idle → Live)



Safeguard:

- Have a fully prepared environment.
- Quick rollback by switching traffic back.



Change safety checklist

- ✓ Backward-compatible API/schema during overlap (e.g. both v1 and v2 work).
- ✓ Keep a known artifact/version for easy rollback.
- ✓ Data migrations are separate from application rollback.
- ✓ Readiness probe before receiving traffic.
- ✓ Configuration and secrets are external (ConfigMap/Secret), not baked into images.
- ✓ Follow change process (review, test, staged rollout).



Important

Rollback changes the workload version — it does NOT automatically undo database writes or migrations.
Use separate migration strategy with backward compatibility.



Lab

Find gateway environment configuration in the manifest.

What would you need for a real zero-downtime rollout beyond the default Deployment behavior?

Multi-Container Applications

A single application can run as multiple containers working together.



Why multiple containers?

- Separate concerns (each container has one job)
- Use the right technology and dependencies
- Scale, update and manage components independently
- Easier to develop, test and deploy

1 Example: Web + Database Application



2 docker-compose.yml (example)

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "8080:8080"
    depends_on:
      - db
    environment:
      - DB_HOST=db
      - DB_PORT=3306
  db:
    image: mysql:8
    environment:
      - MYSQL_ROOT_PASSWORD=example
      - MYSQL_DATABASE=appdb
    ports:
      - "3306:3306"
    volumes:
      - db-data:/var/lib/mysql
volumes:
  db-data:
```

Services

Define multiple containers that work together.

web

- Built from Dockerfile in current dir
- Exposes port 8080
- Depends on db container
- Uses environment variables

db

- Uses official MySQL image
- Exposes port 3306
- Stores data in a named volume

volumes

Persistent storage for database data.

3 Run The Application

```
# Start all containers
docker compose up -d
# Check running containers
docker compose ps
# View logs
docker compose logs -f
# Stop and remove
docker compose down
```

4 Running Containers

```
$ docker compose ps
```

NAME	IMAGE	COMMAND	PORTS
app-web-1	app-web	"python app.py"	0.0.0.0:8080->8080/tcp
app-db-1	mysql:8	"docker-entrypoint..."	0.0.0.0:3306->3306/tcp



Key Takeaways

- Docker Compose lets you run multiple containers as a single application.
- Each container has a specific responsibility.
- Containers can communicate over an internal network (using service names).
- Use volumes for persistent data.



Lab Task

- 1 Use the provided `docker-compose.yml` to run the application.
- 2 Access the web app on `http://localhost:8080`.
- 3 Inspect the containers, logs and network: `docker compose ps` / `docker compose logs`.
- 4 Modify an environment variable and recreate the containers to see the change.

Session 3 Checkpoint

Let's recap — how do all the pieces work together, and what is each responsible for?

1. Code

Your application source code



2. Image

Package the app into a container image



e.g. `app:v1`

3. Container

Running instance of the image



4. Pod

Kubernetes scheduling unit (1 or more containers)



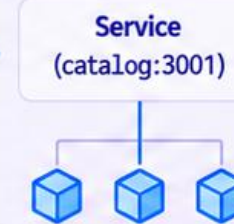
5. Deployment

Maintains desired number and version of Pods (ReplicaSet)



6. Service

Stable network endpoint for changing Pods



Readiness Probe

Should this Pod receive traffic?

- Checks if the application is ready (e.g., dependency connections, warmup).
- If it fails, the Pod is **removed from Service endpoints** (but not restarted).



Liveness Probe

Should this container be restarted?

- Checks if the container is healthy (e.g., responding, not stuck).
- If it fails, Kubernetes **restarts the container**.



Important Note

- Probes are not configured automatically in the starter manifest.
- A running Pod does not mean the application is fully healthy.
- A restarted Pod may lose in-memory data.



What Kubernetes (Platform) Guarantees

- ✓ Schedules Pods on healthy nodes
- ✓ Restarts failed containers
- ✓ Maintains the desired number of replicas
- ✓ Rolls out new versions (rolling update)
- ✓ Provides a stable network endpoint (Service)
- ✓ Routes traffic only to ready Pods



What Kubernetes Does NOT Guarantee

- ✗ Business logic correctness
- ✗ Durable storage of in-memory data
- ✗ Data consistency (e.g., no duplicate orders)
- ✗ Application-level retries or idempotency
- ✗ Database schema compatibility
- ✗ Backup, authorization, or safe data design



What the Application Must Guarantee

- ✓ Correctness of business logic
- ✓ Durable storage (use a database / persistent storage)
- ✓ Idempotency and safe retries
- ✓ Data consistency and validation
- ✓ Compatible API/schema changes
- ✓ Handle partial failures and restarts gracefully



Quick Check

Who is responsible?

Scenario	Responsible
Container crashes	Kubernetes
Pod is not ready	Kubernetes (removes from traffic)
Pod is replaced	Kubernetes
Order disappears after restart	Application
Duplicate order after retry	Application
DB migration breaks old Pods	Application + deployment strategy



Lab Task (Student Output)

Write down:

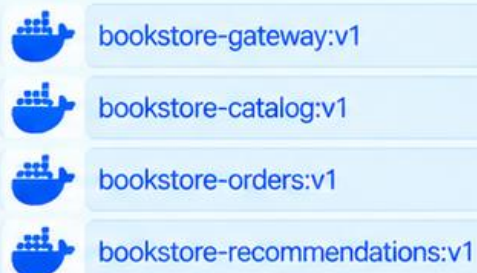
- 1 One thing Kubernetes guarantees.
- 2 One thing Kubernetes does NOT guarantee.
- 3 One application safeguard needed for production.

Deploy The Existing Images

Use pre-built Docker images, deploy to Kubernetes and verify the application is running.

1 Existing Docker Images

Images are already built and available in your Docker host.



2 Kubernetes Deployment

Deploy using the provided manifest (bookstore.yaml).



3 Running Pods

Kubernetes creates Pods using the existing images.



4 Access The Application

Expose the gateway service and access the application.



1. Load Images into Minikube

Use the existing images from your Docker host.

```
minikube image load bookstore-gateway:v1
minikube image load bookstore-catalog:v1
minikube image load bookstore-orders:v1
minikube image load bookstore-recommendations:v1

# Verify images are loaded
minikube image ls | grep bookstore
```

2. Deploy and Check Status

Apply the manifest and verify rollout.

```
kubectl apply -f k8s/bookstore.yaml

# Check rollout status
kubectl rollout status deployment/gateway --timeout=120s
kubectl rollout status deployment/catalog --timeout=120s
kubectl rollout status deployment/orders --timeout=120s
kubectl rollout status deployment/recommendations --timeout=120s

# Check running Pods
kubectl get pods
```

3. Access the Application

Expose the gateway service and open it in browser.

```
kubectl port-forward service/gateway 8080:8080

# In another terminal, access the application
curl -i http://localhost:8080/api/books

# Or open in browser
http://localhost:8080
```



Key Takeaways

- You can deploy existing images without rebuilding them in minikube.
- Kubernetes creates Deployments, ReplicaSets and Pods from the manifest.
- Use `rollout status` to verify successful deployment.
- Access the application via port-forward and check the actual response.



Important Notes

- `imagePullPolicy: Never` is used to pull images from local minikube.
- Ensure the exact image tags match (e.g., v1).
- Check the application response, not just that Pods are Running.
- If an image is missing, inspect the tag rather than repeatedly applying the YAML.
- Do not treat Running alone as user-facing success.



Lab Task

- Deploy the provided manifest using the existing images.
- Watch the Pods appear and verify rollout status.
- Access the application via `http://localhost:8080` and explore the `/api/books` endpoint.

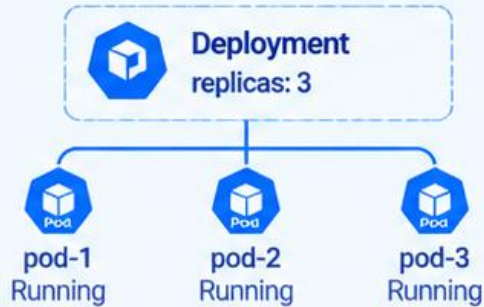
Delete A Managed Pod

Delete a Pod and see how Kubernetes automatically recreates it.

1

Current State

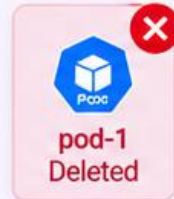
Deployment maintains the desired replicas (e.g., 3).



2

Delete a Pod

We manually delete one of the running Pods.



3

Kubernetes Recreates

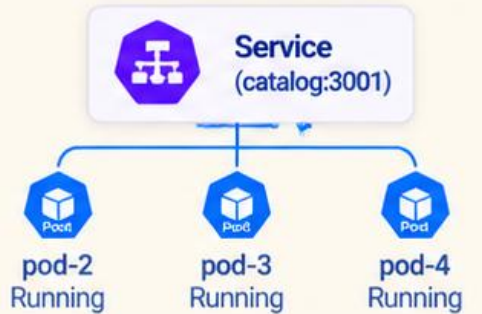
Deployment notices the missing Pod and creates a new one to maintain the desired replicas.



4

Application Stays Available

Service continues to route traffic to the healthy Pods (no downtime).



>_

1. Find a Pod and delete it

Get the pod name and delete it.

```
# Find pod names (example for catalog)
kubectl get pods -l app=catalog

# Delete one pod
kubectl delete pod <pod-name>
```



>_

2. Verify a new Pod is created

Kubernetes automatically creates a new Pod to maintain the desired replicas.

```
# Watch pods
kubectl get pods -l app=catalog -w

# Or check deployment status
kubectl get deployment catalog
```



Key Takeaways

- Deleting a Pod managed by a Deployment is safe.
- Kubernetes **automatically recreates** the Pod to maintain the **desired replicas**.
- Service keeps routing traffic** to the available Pods.
- This provides high availability and self-healing.



Important Note

- This works because the Pod is managed by a **Deployment** (or StatefulSet).
- If you delete a Pod **not managed by a controller**, it will **not be recreated**.
- Kubernetes ensures the desired state, but your application should be stateless or handle restarts gracefully.

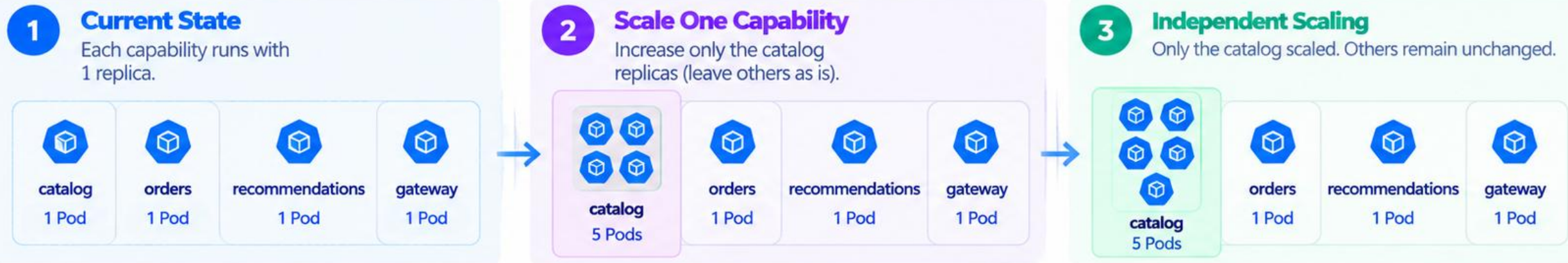


Lab Task

- Get the pod names for the catalog deployment.
- Delete one running Pod.
- Watch the pods and observe a new Pod being created.
- Check the deployment status and confirm the replicas return to the desired count.

Give Capacity To One Capability

Scale one deployment to handle more traffic, without affecting others.



1. Scale the catalog deployment

Increase the number of replicas to 5.

```
# Scale catalog to 5 replicas
kubectl scale deployment catalog --replicas=5

# Check status
kubectl rollout status deployment/catalog --timeout=120s
```



2. Verify the replicas

Check that only catalog has more Pods, others remain the same.

```
# List deployments
kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE
catalog	5/5	5	5
orders	1/1	1	1
recommendations	1/1	1	1
gateway	1/1	1	1



Key Takeaways

- You can scale a single deployment to give more capacity to a capability that needs it.
- Replicas provide higher availability and can handle more load.
- Each capability can be scaled independently.
- Scaling does not automatically increase throughput 5x — it also depends on CPU, memory, network, and downstream systems.



Important Notes

- The catalog is typically a read-only list, so multiple replicas can handle requests safely.
- Minikube is a single node, so this does not show multi-node high availability.
- Do not scale workloads with volatile order data without ensuring correctness and idempotency.
- Monitor resource usage if you scale too high.



Lab Task

- Scale the catalog deployment to 5 replicas.
- Verify the new Pods are running and ready.
- Confirm other deployments (orders, recommendations, gateway) remain at 1 replica.
- (Optional) Scale back to 4 replicas.

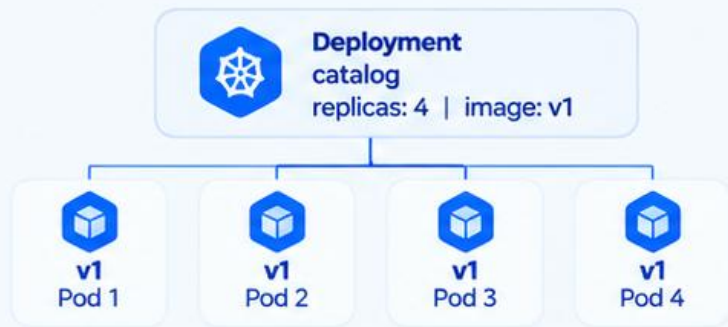
Roll Forward, Then Roll Back

Update to a new version and rollback if needed — with zero downtime.

1

Current State

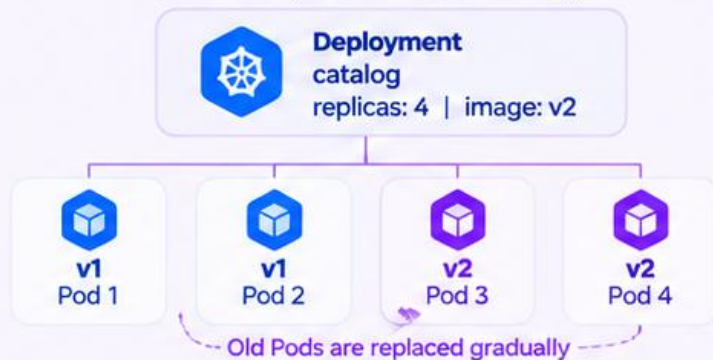
Catalog running with v1



2

Rolling Update

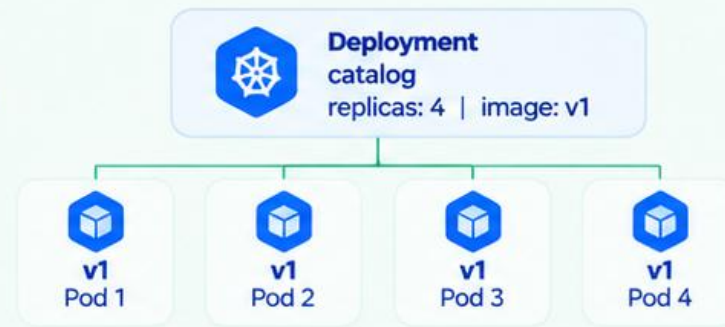
Kubernetes gradually replaces Pods with v2 (maintains availability).



3

Rollback (if needed)

Revert to the previous version (v1).



>_

1. Build and Push New Image (v2)

Build a new image and update the deployment.

```
docker build -t bookstore-catalog:v2 .
minikube image load bookstore-catalog:v2
kubectl set image deployment/catalog catalog=bookstore-catalog:v2
kubectl rollout status deployment/catalog --timeout=120s
curl -I http://localhost:8080/api/books
kubectl rollout undo deployment/catalog
kubectl set image deployment/catalog catalog=bookstore-catalog:v1 --timeout=120s
curl -I http://localhost:8080/api/books
```

>_

2. Check Rollout Status and History

Observe the rollout and rollback details.

```
kubectl rollout status deployment/catalog
kubectl get pods -l app=catalog -w

# View rollout history
kubectl rollout history deployment/catalog
# Rollback to previous version
kubectl rollout undo deployment/catalog
kubectl rollout status deployment/catalog
```



Key Takeaways

- Rolling update replaces Pods gradually, ensuring zero downtime.
- Kubernetes maintains the desired replica count during the update.
- You can roll forward to a new version and roll back if needed.
- Always verify the application response after the rollout.



Important Notes

- Rolling update uses a new ReplicaSet, not in-place changes.
- No need to delete Pods manually.
- Keep readiness probes configured to avoid serving traffic from not yet ready Pods.
- Rollback is fast as the previous version is still available.

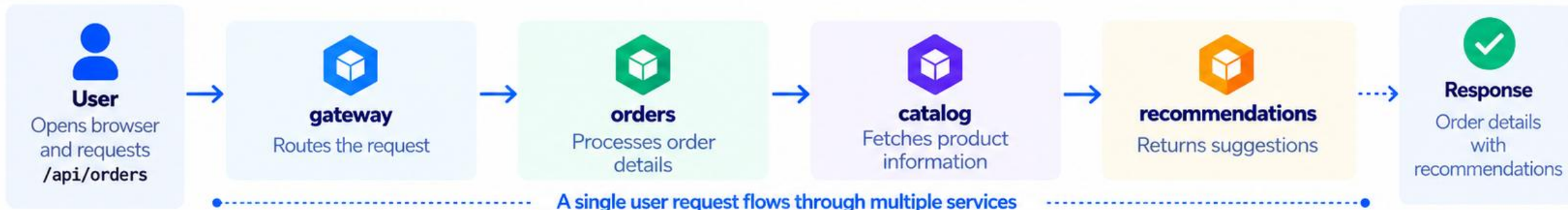


Lab Task

- Build a new image (v2) and update the catalog deployment.
- Watch the Pods getting replaced.
- Verify the application is still accessible.
- Roll back to v1 and confirm the Pods run the old version again.

Observe The User Journey

Trace a request across services and see logs, metrics and traces in action.



1. Logs

What happened at each service?

```
2026-10-01T10:15:23Z gateway INFO Incoming request "/api/orders"
2026-10-01T10:15:23Z orders INFO Processing order for user: 123
2026-10-01T10:15:23Z catalog INFO Fetching product details
2026-10-01T10:15:24Z recommendations INFO Returning 5 items
2026-10-01T10:15:24Z orders INFO Order completed successfully
```



2. Metrics

How is the service behaving?

Request Rate (req/s)



CPU Usage

32%

Memory Usage

512Mi

Request Rate

28 req/s

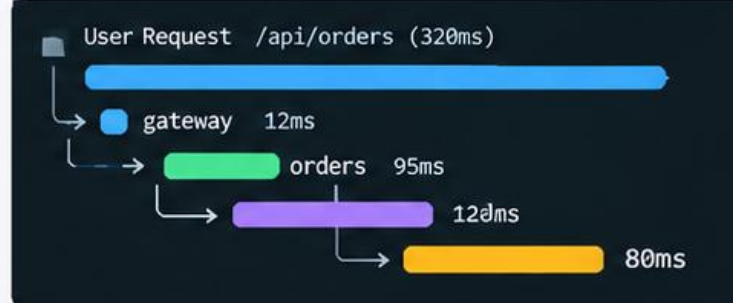
Error Rate

0.2%



3. Traces

Where did the request go and how long did it take?



Run the Lab Commands

Generate load and check logs for each service.

```
# Check logs for orders service
kubectl logs deployment/orders --tail=20

# Check logs for gateway service
kubectl logs deployment/gateway --tail=20

# (Optional) Generate some traffic
for i in {1..5}; do curl -s http://localhost:8080/api/orders > /dev/null; done
```



Key Takeaways

- Logs show what happened at each service.
- Metrics help you understand overall behavior (rate, errors, latency, saturation).
- Traces show the full path and time spent across services.
- This is the starting point of an observability stack.
- A complete setup also sends logs, metrics and traces to tools like Prometheus + Grafana, OpenSearch and OpenTelemetry (Jaeger/Tempo).

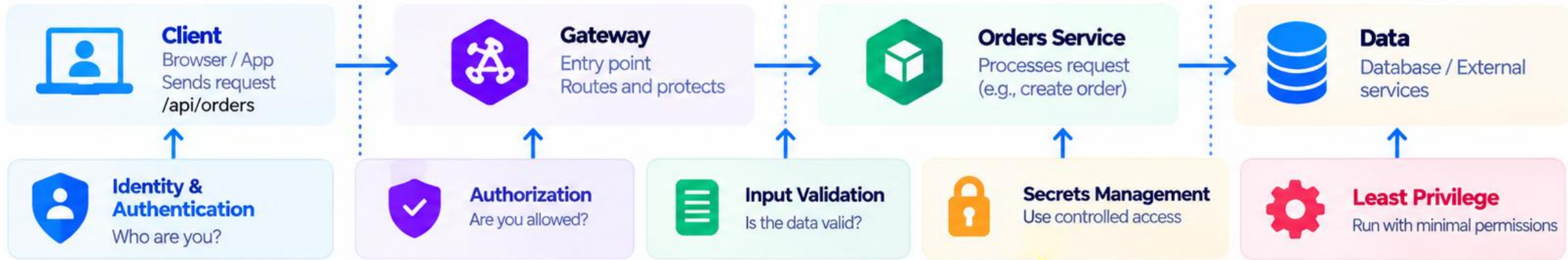


Lab Task (Student Output)

- 1 Send a few requests to `/api/orders`.
- 2 Capture and share logs from orders and gateway services.
- 3 Observe the response time and trace flow across services.
- 4 (Optional) Identify any errors or slow calls and share what you found.

Security Is More Than A Private Port

A private network reduces exposure — it does not prove trust.



Common Gaps vs Production Controls

Starter gap			Production control	
	No identity or authorization	→	✓	Authenticate and check permissions
	Client-supplied order fields	→	✓	Validate against authoritative rules/data
	Containers run as root	→	✓	Run with a restricted user and permissions
	No network policy	→	✓	Restrict allowed flows and credentials

Think Like Production

-  **Authenticate**
Verify who is making the request.
-  **Authorize**
Check if they are allowed to perform the action.
-  **Validate inputs**
Do not blindly trust client-supplied data.
-  **Protect secrets + minimize privileges**
Keep credentials out of source code, images and logs. Run with least privileges.



Private ≠ trusted

A private network or internal cluster reduces exposure, but it does not authenticate users, validate data, or prevent malicious requests.

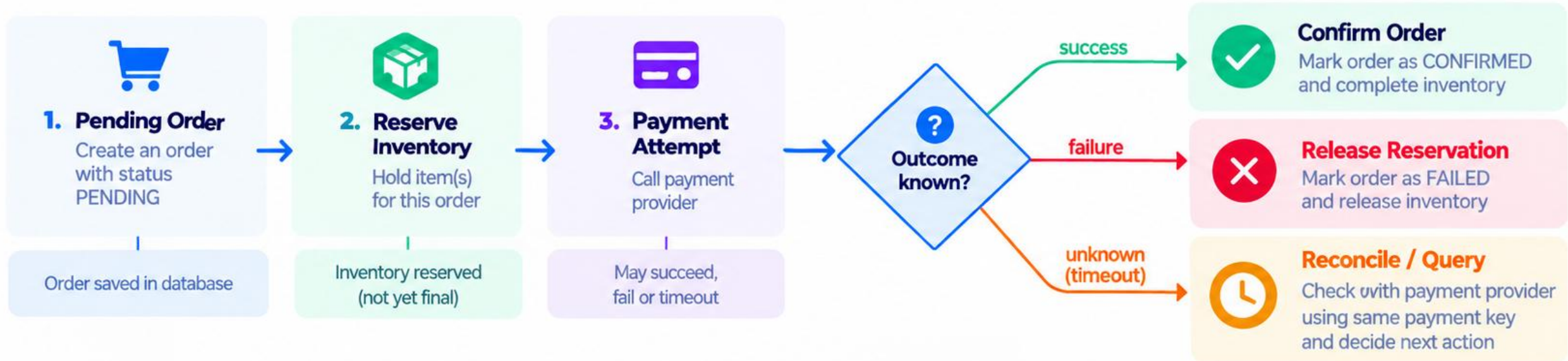


Lab Task

Inspect the order request and name one client-supplied value that the service should not blindly trust.

Checkout Needs Business Recovery

Handle failures gracefully and reconcile data for a consistent user experience.



Key Takeaways

- 1 Checkout is a multi-step process across services.
- 2 Timeouts can leave the outcome unknown.
- 3 Reconciliation is needed to decide retry or compensation.
- 4 Compensation (e.g., refund) is a business action, not an automatic rollback.
- 5 Ensure idempotency and use a unique payment key to safely retry or check status.



Common Failure Scenarios



Payment timeout
User times out but provider captured money.



Partial failure
Inventory reserved but payment failed.



Unknown outcome
Network issue, no response from provider.



Retry or compensate
Query provider with same payment key or trigger refund if needed.



Lab Task

- 1 Place an order through the application.
- 2 Simulate a payment timeout (e.g., delay the payment provider).
- 3 Check the order status and inventory state using logs or API.
- 4 Reconcile by querying the payment provider (with the same payment key).
- 5 Decide the next action: confirm, retry or refund. Observe the final consistent state.

Release A Known Artifact

Test, scan and promote a known artifact through the pipeline.



1. Build & Push a Known Image

Use a pre-built or sample application and push to the registry.

```
# Build the application image
docker build -t bookstore-catalog:v1 .

# Tag and push to a registry (Docker Hub example)
docker tag bookstore-catalog:v1 <your-dockerhub-username>/bookstore-catalog:v1
docker push <your-dockerhub-username>/bookstore-catalog:v1
```



2. Scan and Deploy

Scan the image and deploy to Kubernetes.

```
# Scan the image (Trivy example)
trivy image <your-dockerhub-username>/bookstore-catalog:v1

# Deploy using Kubernetes manifest
kubectl apply -f k8s/bookstore-catalog.yaml

# Check rollout status
kubectl rollout status deployment/bookstore-catalog --timeout=120s

# Verify the application
kubectl get pods
curl -i http://localhost:8080/api/books
```



Key Takeaways

- Artifacts should be tested, scanned and traced before deployment.
- Automate quality, security and policy checks in the pipeline.
- Use container registries with access control.
- Deploy only after successful scans and tests.
- Maintain versioning and immutability for traceability.



Security & Quality Checks

- SAST/DAST:** Scan source code for vulnerabilities.
- SCA:** Check open source dependencies.
- Image scan:** Find vulnerabilities in container image.
- Policy checks:** Ensure only approved artifacts are deployed.



Lab Task

- 1 Build the sample application image.
- 2 Scan the image for vulnerabilities.
- 3 Push the image to a container registry.
- 4 Deploy the application to Kubernetes.
- 5 Verify the application is running and accessible.
- 6 (Optional) View the image in the registry UI.

Industry Case: Reduce Dependency Complexity

Avoid tight coupling and understand the impact of downstream services.

Complex Dependencies



Example Impact



1 order request touches
5+ services
(internal + external)



A slow external service
can increase overall latency



Understanding dependencies
helps set timeouts, retries and
fallbacks at the right places



Explore Dependencies in the Lab

Use logs and simple tools to understand which services are called.

```
# Send a sample request to create an order
curl -i http://localhost:8080/api/orders

# Check logs to see downstream calls
kubectl logs deployment/gateway --tail=50
kubectl logs deployment/orders --tail=50
kubectl logs deployment/payment --tail=50
```



Where to Look in a Real System

Different tools provide different views of dependencies.

	Logs	Which services are called and in what order
	Metrics	Latency, error rate, saturation, queue length
	Traces	Full request path and time spent in each service
	Service map	Visualize dependencies and spot critical paths



Key Takeaways

- A single user request can involve multiple services (internal and external).
- Tight coupling increases latency, failure impact and operational complexity.
- Use logs, metrics and traces to understand dependencies.
- Apply timeouts, retries and fallbacks where needed.
- Consider async patterns or circuit breakers for external services.



Lab Task

- 1 Send a few order requests and observe logs from multiple services.
- 2 Identify which services are called and in what sequence.
- 3 Check if any external service is involved (e.g., payment or shipping).
- 4 Note one improvement to reduce dependency complexity (e.g., async, caching, timeout).

Defend A Checkout Extension

Handle failures gracefully with fallback strategies and clear boundaries.



Key Takeaways

- 1 Define clear interfaces and boundaries for each extension.
- 2 Keep critical and optional components separate.
- 3 Use timeouts, retries and fallback strategies.
- 4 One failing extension should not block the entire checkout.
- 5 Monitor and add circuit breakers where needed.



Example Failure Case

Recommendation service times out (2s) during checkout.



Design Considerations

- ✓ Which extensions are critical vs optional?
- ✓ What timeout and retry policy should we use?
- ✓ How to handle partial failures and invalid responses?
- ✓ Where to persist and reconcile data if needed?
- ✓ How to monitor and alert failures?

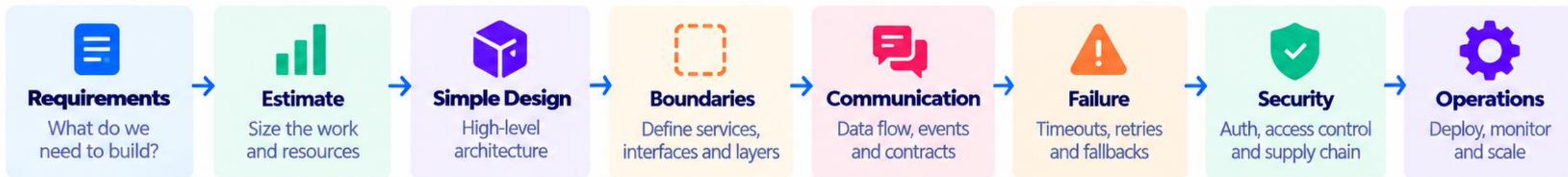


Lab Task

- 1 Draw a simple checkout flow with 3 extensions: inventory, payment, recommendations.
- 2 Mark which ones are critical and optional.
- 3 Add a failure case (e.g., recommendations times out).
- 4 Show how the system continues checkout with fallback.
- 5 Identify what to log and monitor.

Design, Demonstrate, Explain

Bring the whole system together — and justify every decision.



What we built

From a monolith to a resilient, scalable system.



What we proved

- ✓ How to package and deploy the application
- ✓ How Pods are replaced automatically
- ✓ How one service can scale
- ✓ How traffic is routed through the gateway
- ✓ How failures are contained with retries/fallbacks



What we did not prove

- ✗ Production-safe checkout (end-to-end)
- ✗ Durable business state and data consistency
- ✗ Zero-downtime guarantees
- ✗ Full observability and security in production



Reflect



What would Kubernetes NOT solve for your application?



Defend one design decision in 30 seconds.



Next



Design assignment:
Reviews Service

Requirements + estimate + architecture + failure policy



Key Takeaway

Good system design is not about adding technologies — it is about **making and defending the right decisions**.

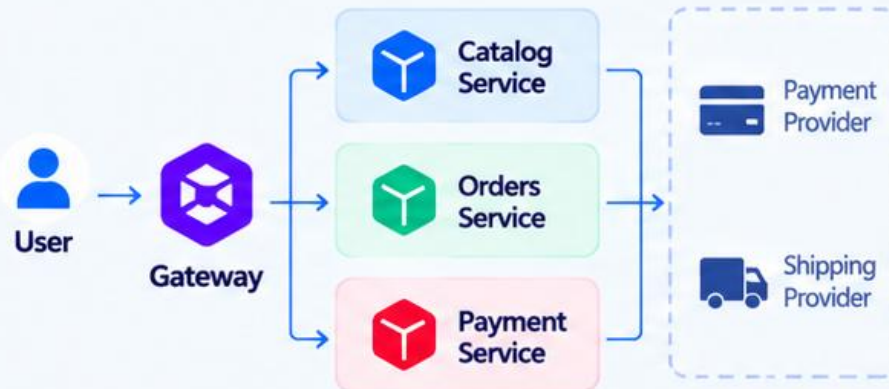
Session 4 Checkpoint

Let's recap what we built, the key learnings, and what's next.

What We Covered

- 1 Using containers and Kubernetes for our services
- 2 Observability: logs, metrics and user journey
- 3 Security across the stack
- 4 Handling failures and business recovery
- 5 CI/CD and releasing a known artifact
- 6 Reducing dependency complexity
- 7 Designing a checkout extension

Application we built



Key Takeaways

- ✓ Break systems into clear services with well-defined interfaces.
- ✓ Use observability to understand user impact.
- ✓ Security is more than a private port.
- ✓ Design for failures: timeouts, retries, idempotency and reconciliation.
- ✓ Release safely with automated checks and rollbacks.



Common Challenges



Timeouts and partial failures (e.g., payment succeeded but order not created).



Complex dependencies increasing latency and cost.



Inconsistent data without reconciliation.



Security gaps in source code, dependencies or images.



Rolling out changes without affecting users.



What Kubernetes did NOT solve



Business workflow and data consistency.



Application-level timeouts, retries and idempotency.



Designing the right service boundaries and dependencies.



What's Next?

- 1 Implement the Reviews Service assignment.
- 2 Write requirements, estimate and design.
- 3 Add one failure case and recovery policy.
- 4 Justify key design decisions and trade-offs.
- 5 Prepare a short presentation (diagram + 30 seconds explanation).

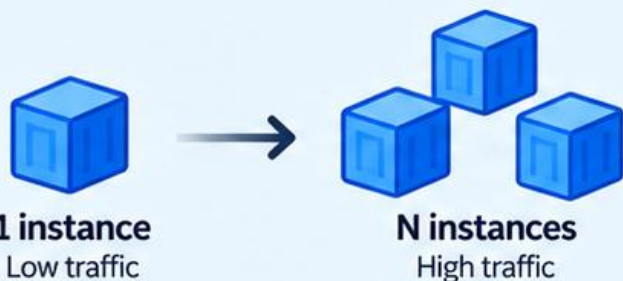
Scalability, Reliability and Resiliency

Key qualities for building production-ready, cloud-native systems



Scalability

Ability to handle growing load by adding resources.



- Scale up (bigger instances) or scale out (more instances)
- Handle increased users/requests
- Use auto scaling (e.g., Kubernetes HPA)
- Keep performance consistent



Reliability

Ability to deliver expected functionality consistently over time.



- Minimize downtime and failures
- Use redundancy (multiple instances, multi-AZ/region)
- Health checks and automatic recovery
- Monitoring, alerting and backups



Resiliency

Ability to recover quickly from failures and continue serving users.



- Graceful handling of failures
- Automatic restart / rescheduling
- Fallback and retry mechanisms
- Fault isolation (one service failure does not bring down the whole system)



Together, **Scalability**, **Reliability** and **Resiliency** ensure the application can handle more users, stay available and recover from failures — even in real-world conditions.

Top 10 System Design Interview Questions

Practice these to build a strong foundation in scalable system design.

1 Design a URL Shortener

Convert long URLs into short, unique and scalable links.

`https://example.com/very/long/url`



`https://short.ly/ab3x`

- Short code generation (Base62)
- High availability & low latency
- Handle collisions and duplicates
- Analytics (clicks, referrers)
- Custom alias, expiry (optional)

2 Design a Rate Limiter

Control the number of requests from users or services.



- Token bucket / Sliding window
- Distributed implementation
- Low latency and high throughput
- Handle burst traffic
- Per user / per API / per service

3 Design a Chat Application

Real-time messaging between users at scale.



- Real-time communication (WebSockets)
- Message delivery and ordering
- Presence, online status
- Store chat history
- Group chats and media (optional)

4 Design Instagram

Share photos and videos with followers.



- Upload and store media (object storage)
- Feed generation (fan-out, fan-in)
- Caching and CDN
- Likes, comments, follows
- Scalable read-heavy system

5 Design Twitter (X)

Post short messages and build timelines at scale.



- Tweet ingestion and storage
- Timeline generation (fan-out)
- Followers and follow graph
- Trending topics and search
- Handle high write and read traffic

6 Design an E-commerce System

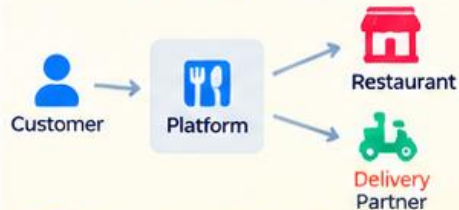
Online store for products and orders.



- Product catalog and search
- Cart and order management
- Inventory and stock consistency
- Payment integration
- Scalability during sales (e.g., BBD)

7 Design a Food Delivery System

Connect customers, restaurants and delivery partners.



- Restaurant search and menu
- Order placement and tracking
- Real-time location updates
- Matching and assignment
- Handle peak traffic and cancellations

8 Design a Ticket Booking System

Book seats for events, movies or travel.



- Seat inventory and real-time locking
- Handle high concurrency
- Prevent double booking
- Reservations with expiry
- Scaling for popular events

9 Design a Payment System

Process secure and reliable payments at scale.



- Secure payment processing
- Idempotency and retries
- Handle failures and reconciliation
- Fraud detection and risk checks
- High availability and audit logs

10 Design Uber (Ride-Hailing)

Match riders with drivers in real-time.



- Real-time location and matching
- Geospatial indexing and search
- Driver-rider coordination
- Surge pricing and dynamic demand
- High availability and low latency

Thank You!

For your time and attention.

Questions | Discussions | Keep Building

Ravi Sajjanar

Cloud Developer - 2

HPE

